Università degli Studi di Padova

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Department of Information Engineering

Master's Degree in
Control Systems Engineering

Kinodynamic RRT for path planning of a small fixed-wing UAV*

Supervisor:
Prof. Ruggero Carli

Co-Supervisor:
Prof. Giuseppe Loianno

Graduand:
Stefano Lacovara

Academic Year 2024/2025
Graduation Date April 15, 2025

Contents

Introduction **1**

1 Fixed-Wing Aircraft Model **3**
1.1 Coordinate Frames 4
1.1.1 Inertial Frame $\mathcal{F}^i$ 4
1.1.2 Vehicle Frame $\mathcal{F}^v$ 5
1.1.3 Body Frame $\mathcal{F}^b$ 7
1.1.4 Stability Frame $\mathcal{F}^s$ 8
1.1.5 Wind Frame $\mathcal{F}^w$ 9
1.2 Kinematic and Dynamic Equations 10
1.2.1 Wind Triangle 11
1.2.2 Velocity Equations 14
1.2.3 Velocity Dynamic 15
1.2.4 Flight-path Angle Dynamic 16
1.2.5 Course Angle Dynamic 17
1.3 Non-linear Model 19
1.4 Feedback Linearization 19
1.5 Constraints 24
1.5.1 Kinematic Constraints 25
1.5.2 Dynamic Constraints 27

2 Path Planning Algorithm **29**
2.1 Rapidly-exploring Random Tree 29
2.2 Rapidly-exploring Random Tree Star 31
2.3 Kinodynamic Rapidly-exploring Random Tree Star 34
2.3.1 Problem Formulation 36
2.3.2 Optimal Control Law 38
2.3.3 Optimal Arrival Time 40
2.3.4 Optimal Trajectory 42
2.3.5 Algorithm 43
2.4 Environmental Constraints 45
2.4.1 Costmap Definition 46
2.4.2 Obstacles 47
2.5 Implementation Considerations 49
2.5.1 Companion Matrix for Polynomial Root-Finding 49
2.5.2 Nilpotency of the State Transition Matrix 53

2.5.3 Breadth-First Search and Depth-First Search 55

3 Results and Analysis **59**
3.1 Matlab Simulations 60
3.2 ARPL Architecture 65
3.2.1 Simulations 70
3.2.2 Real Worlds Test 74

Conclusions **79**

Bibliography **81**

Abstract

This work presents an enhanced version of the Kinodynamic Rapidly-exploring Random Tree (RRT) algorithm, specifically designed for online path planning of small fixed-wing unmanned aerial vehicles (UAVs) operating in environments constrained by obstacles. Unlike conventional RRT-based methods, which primarily focus on geometric feasibility, this approach integrates both dynamic and kinematic constraints of fixed-wing aircraft, directly into the sampling and optimization process. The proposed algorithm leverages a cost-to-go heuristic alongside a rewiring mechanism, ensuring asymptotic optimality while ensuring that the generated paths remain dynamically feasible throughout the planning process. Extensive simulations are conducted to validate the algorithm's effectiveness, demonstrating its ability to generate collision-free, smooth, and dynamically feasible trajectories in complex 3D environments with varying obstacle densities. These simulations highlight improvements in computational efficiency and path quality compared to traditional methods. Furthermore, the algorithm's real-world applicability is validated through flight experiments on a small fixed-wing UAV platform. The experiments confirm the robustness of the algorithm, as it successfully generates and executes dynamically feasible trajectories. The findings presented in this work offer a contribution to the field of autonomous navigation for fixed-wing UAVs, enhancing their ability to perform in obstacle-rich environments. The proposed algorithm has broad potential applications in areas such as surveillance, environmental monitoring, and disaster response, where reliable and efficient path planning is crucial for mission success.

Sommario

Questo lavoro presenta una versione avanzata dell'algoritmo Kinodynamic Rapidly-exploring Random Tree (RRT), progettata per la pianificazione online della traiettoria di piccoli aerei droni ad ali fisse a guida autonoma che operano in ambienti vincolati da ostacoli. A differenza dei metodi convenzionali basati su RRT, che si concentrano principalmente su considerazioni geometriche, questo approccio integra direttamente nei processi di selezione e ottimizzazione i vincoli dinamici e cinematici dei droni ad ali fisse. L'algoritmo proposto sfrutta un'euristica applicata al cost-to-go insieme a un meccanismo di rielaborazione, garantendo ottimalità asintotica e assicurando che le traiettorie generate rimangano cinematicamente e dinamicamente praticabili durante l'intero processo di pianificazione. Sono state condotte diverse simulazioni per validare l'efficacia dell'algoritmo, dimostrando la sua capacità di generare traiettorie prive di collisioni, continue e in grado di soddisfare vincoli fisici in ambienti 3D complessi a densità variabile di ostacoli. Queste simulazioni evidenziano miglioramenti nell'efficienza computazionale e nella qualità della traiettoria rispetto a metodi tradizionali. Inoltre, l'applicabilità dell'algoritmo nel mondo reale è stata validata tramite esperimenti di volo con un drone ad ali fisse di piccole dimensioni. Gli esperimenti confermano la robustezza dell'algoritmo in grado di generare ed inseguire traiettorie fisicamente fattibili. I risultati presentati in questo lavoro offrono un contributo nel campo della navigazione autonoma per i droni ad ali fisse, migliorando la loro capacità di operare in ambienti con ostacoli. L'algoritmo proposto ha ampie applicazioni in settori come la sorveglianza, il monitoraggio ambientale e la risposta a disastri, dove una pianificazione affidabile ed efficiente della traiettoria è cruciale per il successo delle missioni.

Introduction

Fixed-wing Unmanned Aerial Vehicles (UAVs) have become integral to a wide range of applications due to their efficiency, endurance, and ability to cover large distances at high speeds. Unlike rotary-wing UAVs, which provide vertical takeoff and landing capabilities, fixed-wing UAVs excel in missions that require extended flight times, greater payload capacities, and stable operation in challenging environmental conditions. These advantages make fixed-wing UAVs particularly well-suited for critical applications such as aerial surveying, environmental monitoring, border surveillance, precision agriculture, and search-and-rescue operations. For example, in agriculture, fixed-wing UAVs enable large-scale crop monitoring, thereby facilitating more efficient resource utilization. In search-and-rescue missions, their extended range and endurance enable rapid assessments of disaster-stricken areas, improving response coordination and aiding in the timely detection of survivors.

Path planning is a crucial aspect of fixed-wing UAV operations, ensuring that the UAV can navigate efficiently while adhering to both kinematic and dynamic constraints. A variety of path planning algorithms have been developed to address these challenges, typically falling into two main categories: deterministic and randomized approaches. Deterministic algorithms such as A* [1] and D* [2] offer solutions that integrate heuristics to optimize pathfinding or adapt to dynamic environments. However, these algorithms do not inherently consider the vehicle's dynamics, making them less suitable for applications involving fixed-wing UAVs. The Artificial Potential Field (APF) [3] method, while effective in some scenarios, struggles with issues like local minima and does not naturally generate smooth, flyable trajectories suitable for fixed-wing UAVs.

Randomized algorithms such as Probabilistic Roadmap (PRM) [4] and Rapidly-exploring Random Tree (RRT) [5] offer more flexibility in handling complex environments. PRM constructs a connectivity graph that represents feasible paths, but its preprocessing phase limits its applicability for online decision-making. RRT, although efficient in high-dimensional spaces, does not guarantee optimal or smooth paths, which are essential for stable flight. The Rapidly-exploring Random Tree Star (RRT*) [6] improves upon RRT by ensuring asymptotic optimality, although it still requires additional post-processing to refine paths into smooth, feasible trajectories.

Despite their effectiveness in general motion planning, these traditional methods often fail to adequately account for the specific kinematic and dynamic

constraints of fixed-wing UAVs, such as the minimum turning radius and velocity limitations. Furthermore, many of these methods generate paths that exhibit discontinuities, which are unsuitable for maintaining stable flight. To address these issues, various techniques have been proposed to improve path smoothness. For instance, the Theta* [7] algorithm incorporates clothoid-based smoothing to refine trajectory curvature, while Dubins paths [8] provide curvature-constrained solutions tailored for fixed-wing UAVs. However, these methods do not guarantee dynamic feasibility in all cases. Additionally, B-spline trajectories [9] offer smooth, parameterized paths that enhance flight stability, but they require careful tuning to strike the right balance between feasibility and optimality.

The Kinodynamic RRT* (K-RRT*) [10] algorithm addresses these limitations by directly integrating the kinematic and dynamic constraints into the planning process. This approach ensures that the generated paths are not only feasible but also smooth, which is essential for fixed-wing UAVs that rely on continuous, curvature-constrained trajectories for stable flight. By incorporating optimal control principles, K-RRT* enhances the smoothness of the path, addressing the challenges posed by discontinuities and ensuring that the generated trajectory is dynamically feasible.

The structure of the report is designed to explore these concepts in detail. The first section develops the mathematical representation of fixed-wing UAV dynamics, providing an understanding of the coordinate frames, kinematic and dynamic equations, feedback linearization, and flight constraints. The next section examines various path planning techniques, with a particular focus on K-RRT* and its integration of optimal control strategies. Finally, the results and analysis section presents MATLAB simulation results and simulations, as well as real-world validations, using the ARPL architecture, assessing the effectiveness of the proposed methodology.

By addressing both theoretical and practical challenges in fixed-wing UAV path planning, this work aims to contribute to the development of robust, efficient, and dynamically feasible navigation strategies for autonomous aerial vehicles, further advancing the field of UAV motion planning and control.

1 Fixed-Wing Aircraft Model

The development of navigation, guidance, and control strategies for micro aerial vehicles (MAVs) fundamentally relies on the formulation of mathematical models. These models capture the complex motion characteristics of MAVs by describing their dynamics, forming the foundation for control design and path planning.

A comprehensive understanding of MAV dynamics requires the use of multiple coordinate systems, each offering a unique perspective on the vehicle's motion. The inertial frame, which serves as the reference frame for global positioning, is essential for defining the vehicle's position and velocity relative to a fixed, earth-centered coordinate system. Additionally, the body frame, which is fixed to the MAV, is crucial for describing its orientation and velocity relative to its own axes. To account for environmental factors such as wind, the wind frame and stability frame are also considered, providing necessary insights into the effect of external forces on the MAV's motion [8].

In the following sections, the derivation of the nonlinear equations that describe the MAV's dynamics will be presented [8]. These equations model the complex interactions between the vehicle's control inputs, forces, and motion. As MAVs typically operate in highly dynamic environments, precise modeling is critical for ensuring stability and performance during various phases of flight. The nonlinear nature of these equations, however, can complicate analysis and control design. To address this challenge, an input-output feedback linearization technique will be employed. This technique allows for the exact linearization of the system, simplifying the analysis and enabling the application of well-established theoretical framework [11].

Furthermore, a discussion on the kinematic and dynamic constraints will be provided, addressing the limitations imposed by the MAV's physical and aerodynamic properties. These constraints must be carefully considered during the design of control and navigation strategies to ensure safe and efficient operation [8].

For the purposes of this work, a flat, non-rotating Earth assumption is adopted. This approximation, while simplifying the mathematical formulation, remains widely accepted in MAV applications. It provides a reasonable level of accuracy for practical implementations, ensuring that the resulting model is sufficiently precise for real-world applications [8].

Lastly, throughout this chapter, the dependence on time is omitted for clarity and conciseness.

1.1 Coordinate Frames

To analyze and understand the dynamic behavior of MAVs, several coordinate systems are of interest. This section defines and describes the following frames of reference: the inertial frame, the vehicle frame, the vehicle-1 frame, the vehicle-2 frame, the body frame, the stability frame, and the wind frame. The inertial and vehicle frames are related through a translation, while the remaining frames are defined through rotations.

The relative orientations of the vehicle, vehicle-1, vehicle-2, and body frames are characterized by the roll, pitch, and yaw angles, which collectively describe the attitude of the aircraft. These angles are commonly referred to as Euler angles. The relative orientations between the body, stability, and wind frames are defined by the angle of attack and the sideslip angle.

For the purposes of the analysis, it is assumed that the Earth is flat and non-rotating, which serves as a valid approximation for MAVs operating at relatively small altitudes.

1.1.1 Inertial Frame $\mathcal{F}^i$

The inertial coordinate system is an Earth-fixed reference frame with its origin at the defined home location. As illustrated in Figure 1, the unit vector $\mathbf{i}^i$ is directed toward the north, $\mathbf{j}^i$ is directed toward the east, and $\mathbf{k}^i$ is directed toward the center of the Earth, or downward. This coordinate system is commonly referred to as the north-east-down (NED) reference frame. In this system, the north direction is typically associated with the inertial x-axis, the east direction with the inertial y-axis, and the downward direction with the inertial z-axis.

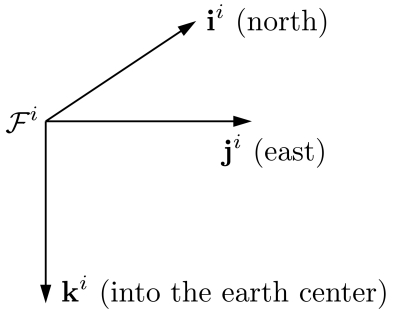

Figure 1: The inertial coordinate frame. The $\mathbf{i}^i$-axis points north, the $\mathbf{j}^i$-axis points east and the $\mathbf{k}^i$-axis points into the earth.

1.1.2 Vehicle Frame $\mathcal{F}^v$

The origin of the vehicle frame is located at the center of mass of the MAV. The axes of the vehicle frame $\mathcal{F}^v$ are aligned with those of the inertial frame $\mathcal{F}^i$. Specifically, as depicted in Figure 2, the unit vector $\mathbf{i}^v$ points toward the north, $\mathbf{j}^v$ points toward the east, and $\mathbf{k}^v$ points toward the center of the Earth.

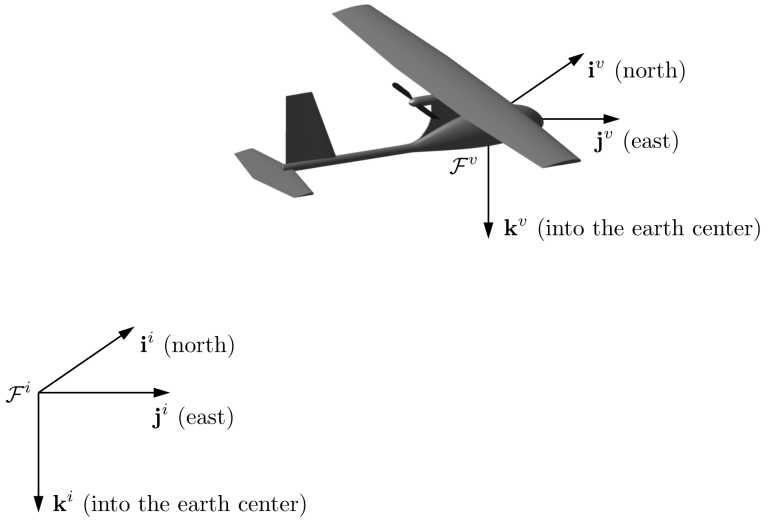

Figure 2: The vehicle coordinate frame. The $\mathbf{i}^v$-axis points north, the $\mathbf{j}^v$-axis points east and the $\mathbf{k}^v$-axis points into the earth.

This frame is essentially a translation of the inertial frame, with the same orientation but shifted in space to the center of mass of the MAV.

Vehicle-1 Frame $\mathcal{F}^{v1}$

The vehicle-1 frame is illustrated in Figure 3, with its origin coinciding with that of the vehicle frame, located at the center of mass of the aircraft. However, the vehicle-1 frame $\mathcal{F}^{v1}$ is rotated in the positive right-handed direction about the axis $\mathbf{k}^v$ by the heading (or yaw) angle ψ. In the absence of additional rotations, the unit vector $\mathbf{i}^{v1}$ is directed along the nose of the airframe, $\mathbf{j}^{v1}$ is directed along the right wing, and $\mathbf{k}^{v1}$ remains aligned with $\mathbf{k}^v$, pointing downward toward the Earth.
The transformation from the vehicle frame $\mathcal{F}^v$ to the vehicle-1 frame $\mathcal{F}^{v1}$ is represented by the following rotation matrix:

$$\mathcal{R}_v^{v1}(\psi) = \begin{bmatrix} \cos\psi & \sin\psi & 0 \\ -\sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

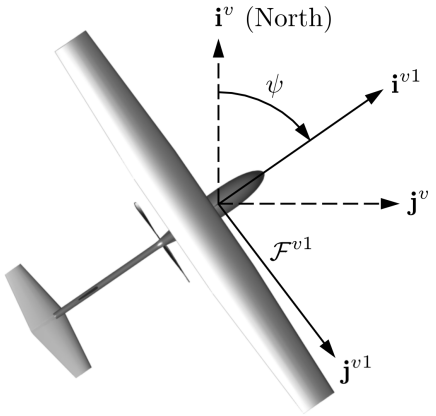

Figure 3: The vehicle-1 frame. The $\mathbf{i}^{v1}$-axis points out the nose of the aircraft, the $\mathbf{j}^{v1}$-axis points out the right wing, and the $\mathbf{k}^{v1}$-axis points into the earth.

Vehicle-2 Frame $\mathcal{F}^{v2}$

The origin of the vehicle-2 frame coincides again with the center of mass of the aircraft. This frame is derived by applying a right-handed rotation to the vehicle-1 frame about the $\mathbf{j}^{v1}$ axis by the pitch angle θ, as shown in Figure 4. The unit vector $\mathbf{i}^{v2}$ is directed along the nose of the aircraft, $\mathbf{j}^{v2}$ is directed along the right wing, and $\mathbf{k}^{v2}$ is directed toward the belly of the aircraft.

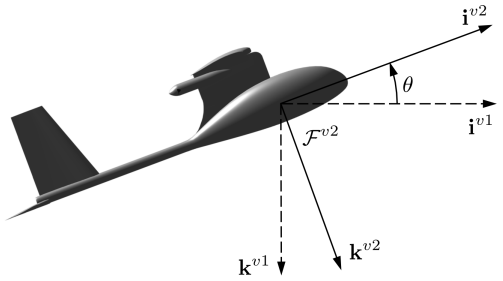

Figure 4: The vehicle-2 frame. The $\mathbf{i}^{v2}$-axis points out the nose of the aircraft, the $\mathbf{j}^{v2}$-axis points out the right wing, and the $\mathbf{k}^{v2}$-axis points out the belly.

The transformation from the vehicle-1 frame $\mathcal{F}^{v1}$ to the vehicle-2 frame $\mathcal{F}^{v2}$ is given by the following rotation matrix:

$$\mathcal{R}_{v1}^{v2}(\theta) = \begin{bmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{bmatrix}$$

1.1.3 Body Frame $\mathcal{F}^b$

The body frame is derived by applying a right-handed rotation to the vehicle-2 frame about the $\mathbf{i}^{v2}$ axis by the roll angle ϕ. The origin of this frame is located at the aircraft's center of mass, with the unit vector $\mathbf{i}^b$ pointing along the nose of the airframe, $\mathbf{j}^b$ pointing along the right wing, and $\mathbf{k}^b$ pointing downward, toward the belly of the aircraft. The body frame is illustrated in Figure 5. The directions of the unit vectors $\mathbf{i}^b$, $\mathbf{j}^b$, and $\mathbf{k}^b$ are typically referred to as the body x-axis, body y-axis, and body z-axis, respectively.

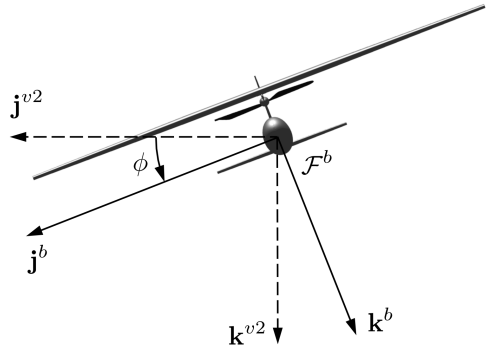

Figure 5: The body frame. The $\mathbf{i}^b$-axis points out the nose of the airframe, the $\mathbf{j}^b$-axis points out the right wing, and the $\mathbf{k}^b$-axis points out the belly.

The transformation from the vehicle-2 frame $\mathcal{F}^{v2}$ to the body frame $\mathcal{F}^b$ is represented by the following rotation matrix:

$$\mathcal{R}^b_{v2}(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & \sin\phi \\ 0 & -\sin\phi & \cos\phi \end{bmatrix}$$

When all prior transformations are combined, the resulting formula for the transformation from the vehicle frame $\mathcal{F}^v$ to the body frame $\mathcal{F}^b$ is given by:

$$\begin{aligned} \mathcal{R}^b_v(\phi,\theta,\psi) &= \mathcal{R}^b_{v2}(\phi)\mathcal{R}^{v2}_{v1}(\theta)\mathcal{R}^{v1}_v(\psi) \\ &= \begin{bmatrix} c_\theta c_\psi & c_\theta s_\psi & -s_\theta \\ s_\phi s_\theta c_\psi - c_\phi s_\psi & s_\phi s_\theta s_\psi + c_\phi c_\psi & s_\phi c_\theta \\ c_\phi s_\theta c_\psi + s_\phi s_\psi & c_\phi s_\theta s_\psi - s_\phi c_\psi & c_\phi c_\theta \end{bmatrix}. \end{aligned}$$

In the above, $c_\phi \triangleq \cos\phi$ and $s_\phi \triangleq \sin\phi$. The angles ϕ, θ, and ψ are collectively known as Euler angles. These angles provide a convenient and intuitive method for representing the orientation of a rigid body in three-dimensional space. The ψ-θ-ϕ sequence of rotations is widely used in the context of air-

craft, though it is one of several different Euler angle conventions. Euler angles are popular due to their straightforward physical interpretation. However, a known drawback of Euler angles is their susceptibility to a mathematical singularity, which can lead to computational issues. Specifically, when the pitch angle θ reaches $\pm 90°$, the yaw angle becomes undefined, resulting in a phenomenon referred to as gimbal lock.

1.1.4 Stability Frame $\mathcal{F}^s$

Aerodynamic forces arise as the airframe moves through the surrounding air. The relative velocity of the aircraft with respect to the ambient air is referred to as the airspeed vector, denoted $\mathbf{V}_a$. The magnitude of this vector is simply termed the airspeed, V_a. To generate lift, the wings of the airframe must be inclined at a positive angle relative to the airspeed vector. This inclination is known as the angle of attack and is represented by α. As illustrated in Figure 6, the $\mathbf{i}^s$-axis is aligned with the projection of $\mathbf{V}_a$ onto the plane defined by the $\mathbf{i}^b$ and $\mathbf{k}^b$ axes. The angle of attack is defined as the angle between the $\mathbf{i}^s$ and $\mathbf{i}^b$ axes.

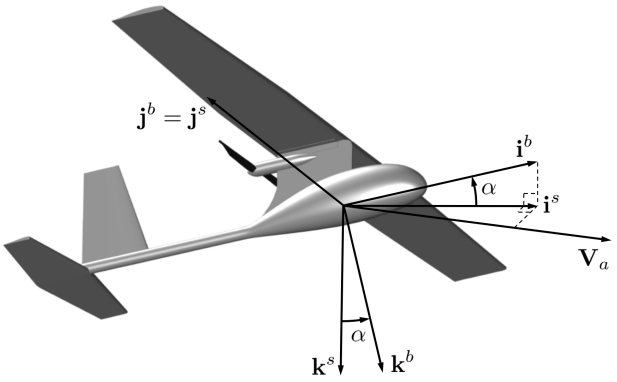

Figure 6: The stability frame. The $\mathbf{i}^s$-axis points along the projection of the airspeed vector onto the $\mathbf{i}^b$-$\mathbf{k}^b$ plane of the body frame, the $\mathbf{i}^s$-axis is identical to the $\mathbf{j}^b$-axis of the body frame, and the $\mathbf{k}^s$-axis is constructed to make a *right*-handed coordinate system. The angle of attack is defined as a right-handed rotation about the body $\mathbf{j}^s$-axis.

The transformation from the stability frame $\mathcal{F}^s$ to the body frame $\mathcal{F}^b$ is given by the following rotation matrix:

$$\mathcal{R}_s^b(\alpha) = \begin{bmatrix} \cos\alpha & 0 & -\sin\alpha \\ 0 & 1 & 0 \\ \sin\alpha & 0 & \cos\alpha \end{bmatrix}$$

1.1.5 Wind Frame $\mathcal{F}^w$

The angle between the airspeed vector and the $\mathbf{i}^b$-$\mathbf{k}^b$ plane is defined as the side-slip angle, denoted by β. As depicted in Figure 7, the wind frame is obtained by applying a right-handed rotation of angle β about the $\mathbf{k}^s$ axis to the stability frame. In this frame, the unit vector $\mathbf{i}^w$ is aligned with the airspeed vector $\mathbf{V}_a$.

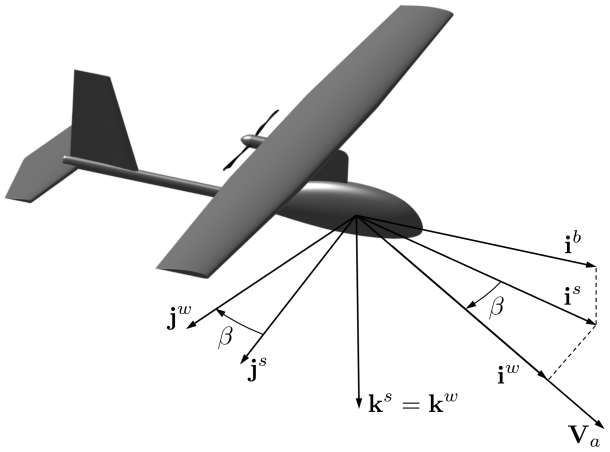

Figure 7: The wind frame. The $\mathbf{i}^w$-axis points along the airspeed vector.

The transformation matrix that relates the stability frame $\mathcal{F}^s$ to the wind frame $\mathcal{F}^w$ is expressed as:

$$\mathcal{R}_s^w(\beta) = \begin{bmatrix} \cos\beta & \sin\beta & 0 \\ -\sin\beta & \cos\beta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

By combining the transformations from the stability frame to the body frame and from the stability frame to the wind frame, the transformation from the body frame $\mathcal{F}^b$ to the wind frame $\mathcal{F}^w$ is given by:

$$\begin{aligned} \mathcal{R}_b^w(\alpha,\beta) &= \mathcal{R}_s^w(\beta)\mathcal{R}_b^s(\alpha) \\ &= \begin{bmatrix} c_\beta c_\alpha & s_\beta & c_\beta s_\alpha \\ -s_\beta c_\alpha & c_\beta & -s_\beta s_\alpha \\ -s_\alpha & 0 & c_\alpha \end{bmatrix}. \end{aligned}$$

where $c_\alpha \triangleq \cos\alpha$ and $s_\alpha \triangleq \sin\alpha$, and similar definitions hold for β.

1.2 Kinematic and Dynamic Equations

Following the discussion of the reference frames, attention is now directed to key definitions relevant to the navigation of a MAV. The motion of a fixed-wing aircraft is inherently influenced by aerodynamic forces and environmental factors. A mathematical formulation is required to describe the aircraft's velocity components and orientation within an inertial reference frame, which serves as the foundation for further navigation and flight path modeling.

The inertial position of the MAV is described in terms of its north, east, and altitude coordinates. Specifically, the position components are given by p_n, which represents the inertial north position of the MAV along $\mathbf{i}^i$ in the inertial reference frame $\mathcal{F}^i$; p_e, which denotes the inertial east position along $\mathbf{j}^i$ in $\mathcal{F}^i$; and h, which corresponds to the inertial altitude measured along the $-\mathbf{k}^i$ axis in $\mathcal{F}^i$. These coordinates collectively define the aircraft's absolute position relative to an earth-fixed frame and serve as the foundation for expressing its velocity and orientation in space.

One aspect of fixed-wing aircraft motion is the wind triangle, which defines the relationship between the aircraft's airspeed, groundspeed, and the effect of wind velocity. The wind triangle offers a means of understanding how wind alters the aircraft's true motion over the ground, serving as the basis for deriving kinematic equations that govern the aircraft's translational motion.

In addition to kinematic considerations, the dynamic behavior of the aircraft is governed by several aerodynamic forces acting on it. These forces include thrust, lift, and drag, each of which plays a significant role in determining the aircraft's flight dynamics. Thrust, denoted as F_{thrust}, is the propulsive force generated by the aircraft's engine. It acts along the aircraft's body-fixed longitudinal axis and provides the necessary acceleration to sustain forward motion. The magnitude of thrust directly affects the aircraft's ability to maintain airspeed and climb rate, as well as the efficiency of flight maneuvers.

Lift, represented as F_{lift}, is the upward force generated by the aircraft's wings as air flows over them. Lift is primarily a result of the pressure difference between the upper and lower surfaces of the wings, caused by the aircraft's forward motion through the air. The flow of air is altered by the ailerons, elevators, and other control surfaces, which influence the distribution of lift and its effectiveness in maintaining flight. The ability to generate sufficient lift is critical for counteracting the weight of the aircraft and maintaining altitude. Lift is also influenced by airspeed and the angle of attack, which determines the strength of the airflow over the wings. Additionally, lift is

indirectly influenced by the thrust since increased thrust can lead to higher airspeed, thereby enhancing lift generation.

Drag, denoted as F_{drag}, is the resistive force that opposes the aircraft's motion through the air. Drag is divided into two components: parasite drag and induced drag. Parasite drag includes form drag, caused by the aircraft's shape and surface friction, and interference drag, which arises from the interaction between different parts of the aircraft. Induced drag, on the other hand, is generated as a result of lift production and increases with the angle of attack and airspeed. Drag is influenced by factors such as the aircraft's velocity, shape, and aerodynamic configuration. Higher airspeed leads to an increase in both parasite and induced drag, requiring greater thrust to maintain the desired velocity.

The relationship between lift, drag, and thrust is interconnected. In the following, the kinematic and dynamic equations governing the motion of the fixed-wing aircraft will be derived. These equations describe the aircraft's velocity components, flight path angle, and course angle, as well as the aerodynamic and environmental factors influencing the aircraft's motion, incorporating the relationships between these forces.

1.2.1 Wind Triangle

When developing the dynamic equations of motion for a MAV, it is essential to recognize that the inertial forces acting on the MAV are determined by velocities and accelerations relative to a fixed (inertial) reference frame. These forces depend on the MAV's movement in relation to this reference frame. In contrast, the aerodynamic forces are influenced by the velocity of the MAV relative to the surrounding air.

In the absence of wind, the airspeed, represented by the velocity relative to the surrounding air $\mathbf{V}_a$, is identical to the groundspeed, which is the velocity relative to the inertial frame $\mathbf{V}_g$. However, since wind is nearly always present, it becomes necessary to distinguish between these two velocities. Airspeed, $\mathbf{V}_a$, and groundspeed, $\mathbf{V}_g$, are no longer equivalent when wind is considered. These velocities are related through the wind velocity $\mathbf{V}_w$, which represents the wind's velocity relative to the inertial frame. The relationship is given by the following expression:

$$\mathbf{V}_a = \mathbf{V}_g + \mathbf{V}_w$$

Understanding this relationship is crucial for accurately describing the MAV's

motion both in the air and relative to the ground, particularly when wind influences its trajectory.

The relationship among the groundspeed vector, the airspeed vector, and the wind vector is commonly referred to as the wind triangle. This geometric representation is crucial for understanding how wind influences an aircraft's trajectory. A more detailed illustration of the wind triangle is provided in two perspectives: the horizontal plane, as shown in Figure 8, and the vertical plane, depicted in Figure 9.

In Figure 8, an aerial vehicle is represented as following a ground track, indicated by the dashed line. The northward direction is defined by the unit vector $\mathbf{i}^i$, while the orientation of the aircraft itself is represented by $\mathbf{i}^b$, which is fixed along the body x-axis. Under conditions of level flight, the heading (yaw) angle ψ, is defined as the angle between $\mathbf{i}^i$ and $\mathbf{i}^b$. This angle specifies the direction in which the nose of the aircraft is pointing, independent of external influences such as wind.

The direction of motion relative to the surrounding air mass is described by the airspeed vector $\mathbf{V}_a$, which accounts for the actual velocity of the aircraft through the air. In steady, level flight, it is common for $\mathbf{V}_a$ to be aligned with $\mathbf{i}^b$, implying that the sideslip angle, denoted as β, is equal to zero. This condition indicates that the aircraft is moving directly forward without any lateral deviation due to wind-induced drift.

The velocity vector $\mathbf{V}_g$ represents the direction in which the vehicle is traveling relative to the ground. The course angle, denoted as χ, is defined as the angle between the inertial north direction and the projection of the inertial velocity vector onto the local north-east plane. This angle determines the actual path of the aircraft over the ground, accounting for any external influences such as wind.

In the presence of a steady, nonzero wind component, the aircraft must adjust its orientation to compensate for wind drift. Specifically, in order to maintain a desired ground track that is not aligned with the wind direction, the aircraft must be oriented at an angle offset from its actual course. This misalignment is referred to as the *crab angle*, denoted as χ_c, and it quantifies the difference between the course angle χ and the heading angle ψ, which represents the direction in which the nose of the aircraft is pointing. The crab angle is mathematically expressed as:

$$\chi_c \triangleq \chi - \psi$$

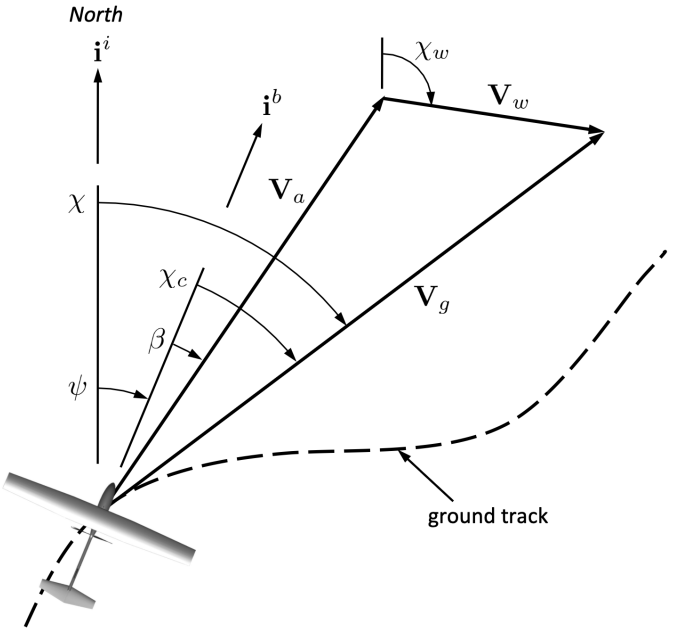

Figure 8: Heading is the direction that the MAV is pointed. Course is the direction of travel relative to the earth's surface. The crab angle is the difference between course and heading. In the absence of wind, the crab angle is zero.

This adjustment ensures that the aircraft compensates for the lateral displacement caused by wind, allowing it to follow the intended ground track.

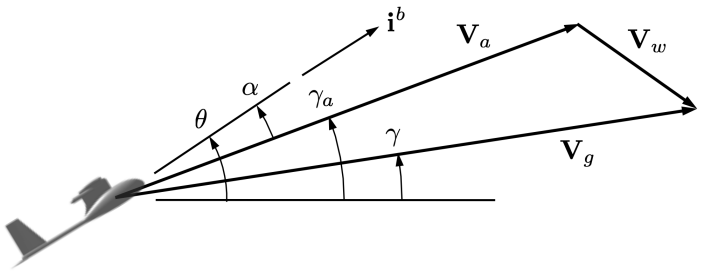

Figure 9: The wind triangle projected onto the vertical plane.

Figure 9 illustrates the vertical component of the wind triangle, which provides insight into the effects of wind on the aircraft's trajectory in the vertical plane. In the presence of a wind component with a downward direction, the angle between the inertial north-east plane and the airspeed vector $\mathbf{V}_a$ is referred to as the air-mass-referenced flight path angle, denoted by γ_a. This angle characterizes the inclination of the aircraft's trajectory relative to the surrounding air mass, independent of ground-referenced effects.
A fundamental relationship exists between the air-mass-referenced flight path

angle, the angle of attack, and the pitch angle. Specifically, the pitch angle θ is the sum of the air-mass-referenced flight path angle and the angle of attack, which can be expressed as:

$$\theta = \gamma_a + \alpha$$

This equation highlights that the aircraft's orientation is the result of both the flight path angle relative to the air mass and the aerodynamic angle of attack. In the absence of wind, the air-mass-referenced flight path angle coincides with the ground-referenced flight path angle, denoted by γ, meaning that the aircraft's trajectory through the air is identical to its trajectory relative to the ground.

1.2.2 Velocity Equations

The orientation of the ground speed vector with respect to an inertial reference frame $\mathbf{V}_g^i$ is characterized by two fundamental angles: the course angle χ and the inertial-referenced flight path angle γ. These angles provide a complete description of the trajectory of the vehicle in both the horizontal and vertical planes. Figure 10 illustrates their definitions in relation to the aircraft's motion.

The flight path angle γ is defined as the inclination of the ground velocity vector $\mathbf{V}_g$ relative to the horizontal plane, representing the vertical component of the aircraft's trajectory. Meanwhile, the course angle χ describes the orientation of the projection of the ground velocity vector onto the horizontal plane, measured with respect to true north. Together, these angles determine the overall trajectory of the vehicle in three-dimensional space.

To mathematically express the ground speed vector in the inertial frame, the following transformation is applied:

$$\begin{aligned}\mathbf{V}_g^i &= \begin{bmatrix}\cos\chi & -\sin\chi & 0\\ \sin\chi & \cos\chi & 0\\ 0 & 0 & 1\end{bmatrix}\begin{bmatrix}\cos\gamma & 0 & \sin\gamma\\ 0 & 1 & 0\\ -\sin\gamma & 0 & \cos\gamma\end{bmatrix}\begin{bmatrix}V_g\\ 0\\ 0\end{bmatrix}\\ &= V_g\begin{bmatrix}\cos\chi\cos\gamma\\ \sin\chi\cos\gamma\\ -\sin\gamma\end{bmatrix}\end{aligned}$$

where V_g is the magnitude of the ground velocity vector, expressed as $V_g = \|\mathbf{V}_g\|$.

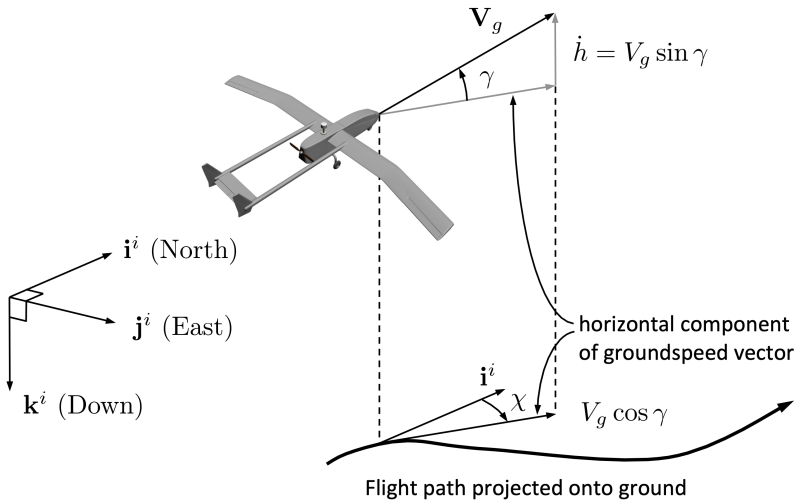

Figure 10: Flight path angle γ and course angle χ.

By utilizing this formulation, the kinematic equations governing the aircraft's motion in the inertial reference frame can be expressed as follows:

$$\begin{bmatrix} \dot{p}_n \\ \dot{p}_e \\ \dot{h} \end{bmatrix} = V_g \begin{bmatrix} \cos\chi\cos\gamma \\ \sin\chi\cos\gamma \\ \sin\gamma \end{bmatrix} \tag{1}$$

These equations describe the time evolution of the aircraft's position in the inertial frame, where $\dot{p}_n$ and $\dot{p}_e$ represent the rates of change of the aircraft's position in the north and east directions, respectively, while $\dot{h}$ represents the rate of change of altitude.

1.2.3 Velocity Dynamic

To determine the rate of change of velocity for a MAV, refer to Figure 11, which illustrates a free-body diagram of the vehicle in a climbing trajectory, characterized by a flight path angle γ and a bank angle ϕ. The forces acting on the MAV are analyzed within the body frame, with particular focus on the forces along the $\mathbf{i}^b$ axis. Newton's second law of motion is then applied to describe the dynamics:

$$m\dot{V}_g = F_{thrust} - F_{drag} - mg\sin\gamma$$

By rearranging this equation to explicitly express the variation of the ground velocity vector, the following formulation is obtained:

$$\dot{V}_g = \frac{F_{thrust}}{m} - \frac{F_{drag}}{m} - g\sin\gamma$$

This equation provides a clear expression for the acceleration of the MAV along the direction of the ground speed vector. The first term on the right-hand side represents the acceleration due to the thrust force, the second term accounts for the drag-induced deceleration, and the third term represents the component of gravitational force acting along the flight path.

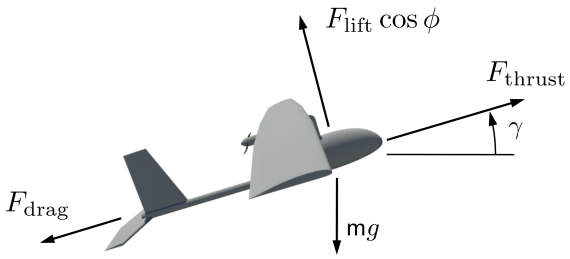

Figure 11: Free-body diagram indicating external forces on the UAV along the i^b axis. The UAV is assumed to be at a roll angle of ϕ.

1.2.4 Flight-path Angle Dynamic

To derive the equations governing the dynamics of the flight-path angle γ, consider a pull-up maneuver in which the aircraft follows a curved trajectory while climbing. The two-dimensional plane $\mathcal{P}$, which is central to this analysis, is defined as the plane containing both the velocity vector $\mathbf{V}_g$ and the vector extending from the center of mass of the aircraft to the instantaneous center of the circular path traced by the aircraft during the pull-up maneuver. A free-body diagram of the MAV in $\mathcal{P}$ is presented in Figure 12.
In order for the aircraft to execute this maneuver, the forces acting on it must satisfy a specific equilibrium condition. This condition can be expressed as:

$$\mathbf{F}_{lift} - \mathbf{F}_{gravity} = \mathbf{F}_{centripetal}$$

As the airframe is banked at an angle ϕ, only the component of the lift force that is projected onto the plane $\mathcal{P}$ contributes to the maneuver. This component is given by $F_{lift}\cos\phi$. The necessary centripetal force required to maintain the circular motion is given by:

$$F_{centripetal} = \frac{mV_g^2}{R} = mV_g\dot{\gamma}$$

By summing the forces in the $\mathbf{i}^b - \mathbf{k}^b$ plane, the following expression for the forces in equilibrium is obtained:

$$F_{\text{lift}} \cos\phi - mg\cos\gamma = mV_g\dot{\gamma}$$

Solving for the rate of change of the flight-path angle $\dot{\gamma}$, the following expression is derived:

$$\dot{\gamma} = \frac{F_{\text{lift}}}{mV_g}\cos\phi - \frac{g}{V_g}\cos\gamma$$

This equation provides the rate of change of the flight-path angle during a maneuver where the aircraft follows a curved trajectory. It accounts for the balance of forces acting on the aircraft, including lift, gravity, and the vehicle's bank angle. The formula shows how these forces, along with the aircraft's velocity, influence the flight-path angle and the aircraft's motion in the vertical plane.

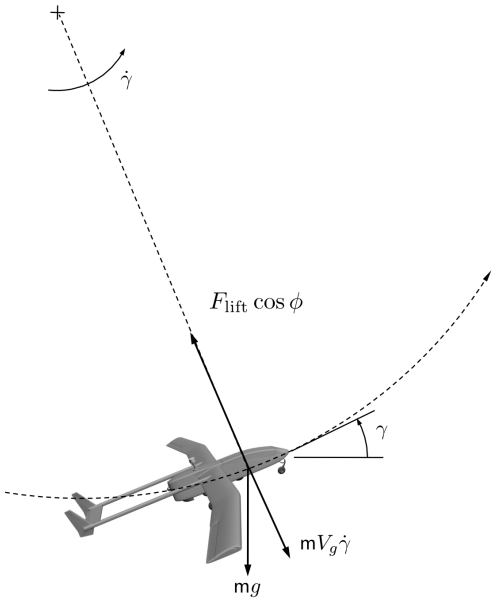

Figure 12: Free-body diagram for a pull-up maneuver. The MAV is at a roll angle of ϕ.

1.2.5 Course Angle Dynamic

In the case of a coordinated turn, the aircraft does not experience lateral acceleration within its body frame. This implies that the vehicle follows a smooth curved trajectory without any skidding or drifting to the side. Instead of exhibiting a lateral slip, the aircraft effectively "carves" through the

turn while maintaining a balance between aerodynamic forces. From a mathematical and analytical standpoint, assuming the turn is coordinated simplifies the relationship between the course rate and the bank angle. This assumption allows for the derivation of a cleaner expression that governs the motion of the vehicle during the turn.

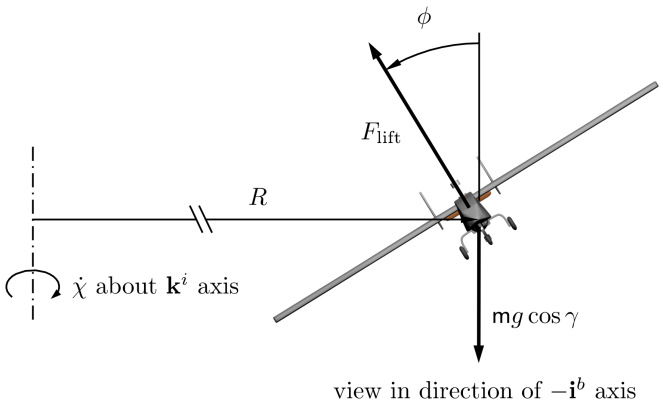

Figure 13: Free-body diagram indicating forces on a MAV in a climbing coordinated turn. The MAV is flying at a flight path angle of γ with the nose of the MAV directed out of the page. The forces shown are in the $\mathbf{j}^b - \mathbf{k}^b$ plane.

During a coordinated turn, the bank angle ϕ is specifically adjusted so that no net lateral forces are present on the MAV. This ensures that the centrifugal force acting on the aircraft is exactly counterbalanced by the horizontal component of the lift force, maintaining equilibrium. As depicted in the free-body diagram shown in Figure 13, this balance can be mathematically represented by the equation:

$$\begin{aligned} F_{lift}\sin\phi &= m\frac{v^2}{R} \\ &= mv\omega \\ &= m(V_g\cos\gamma)\dot{\chi} \end{aligned} \tag{2}$$

Here, the centrifugal force is expressed in terms of the angular rate $\dot{\chi} = \omega$, which corresponds to the rate of change of the course angle about the inertial frame's vertical axis $\mathbf{k}^i$. Additionally, the horizontal component of the groundspeed is represented by $v = V_g\cos\gamma$, where V_g is the total groundspeed and γ is the flight-path angle.

By rearranging the above equation and solving for the course rate, the following expression for $\dot{\chi}$ is obtained:

$$\dot{\chi} = \frac{F_{lift} \sin \phi}{m V_g \cos \gamma}$$

This formula expresses the course rate in terms of the lift force, mass of the vehicle, groundspeed, and flight-path angle. It provides a direct relationship between these parameters during a coordinated turn.

1.3 Non-linear Model

The objective is to derive a path planning algorithm; therefore, several assumptions can be made. The MAV is assumed to remain parallel to the ground during flight, with no wind effects, thereby eliminating any sideslip. By making the approximations $\alpha = \beta = 0$ and $V_w = 0$, several simplifications arise in the system's dynamics. Under these conditions, the wind frame $\mathcal{F}^w$ coincides with the body frame $\mathcal{F}^b$, i.e., $\mathcal{F}^w \equiv \mathcal{F}^b$, and the airspeed vector V_a becomes equivalent to the groundspeed vector V_g, which is commonly denoted as V.
Additionally, these assumptions imply that the pitch angle θ closely approximates the flight path angle γ, and similarly, the yaw angle ψ approximates the course angle χ. These relationships hold under the assumption of zero sideslip angle and no wind, leading to a simpler and more manageable description of the vehicle's motion.
By synthesizing the information derived from the preceding sections, the nonlinear equations governing the motion of the MAV can be expressed as follows:

$$\begin{cases} \dot{p}_n = V \cos \chi \cos \gamma \\ \dot{p}_e = V \sin \chi \cos \gamma \\ \dot{h} = V \sin \gamma \\ \dot{V} = \dfrac{F_{thrust} - F_{drag}}{m} - g \sin \gamma \\ \dot{\gamma} = \dfrac{F_{lift} \cos \phi}{mV} - \dfrac{g \cos \gamma}{V} \\ \dot{\chi} = \dfrac{F_{lift} \sin \phi}{mV \cos \gamma} \end{cases} \tag{3}$$

1.4 Feedback Linearization

In general, dealing with a nonlinear system presents significant challenges due to the inherent complexity of its dynamics. To simplify the problem, it

is possible to apply feedback linearization to the model equations in Eq. 3 by redefining the control inputs as the vector of accelerations:

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} \frac{F_{\text{thrust}} - F_{\text{drag}}}{m} \\ \frac{F_{\text{lift}} \cos\phi}{m} \\ \frac{F_{\text{lift}} \sin\phi}{m} \end{bmatrix}$$

where a_1 represents the tangential acceleration, a_2 corresponds to the vertical component of the normal acceleration, and a_3 denotes the horizontal component of the normal acceleration.

By substituting these inputs into the system dynamics, the equations governing the nonlinear system take the form:

$$\begin{cases} \dot{p}_n = V \cos\chi \cos\gamma \\ \dot{p}_e = V \sin\chi \cos\gamma \\ \dot{h} = V \sin\gamma \\ \dot{V} = -g \sin\gamma + a_1 \\ \dot{\gamma} = -\dfrac{g \cos\gamma}{V} + \dfrac{a_2}{V} \\ \dot{\chi} = \dfrac{a_3}{V \cos\gamma} \end{cases}$$

The nonlinear system, expressed in terms of the state vector $\bar{\mathbf{x}} = \begin{bmatrix} p_n & p_e & h & V & \gamma & \chi \end{bmatrix}^T$, can be rewritten in control-affine form as:

$$\dot{\bar{\mathbf{x}}} = f(\bar{\mathbf{x}}) + \sum_{i=1}^{3} g_i(\bar{\mathbf{x}})\, a_i,$$

where the drift dynamics are given by:

$$f(\bar{\mathbf{x}}) = \begin{bmatrix} V \cos\gamma \cos\chi \\ V \cos\gamma \sin\chi \\ V \sin\gamma \\ -g \sin\gamma \\ -\frac{g \cos\gamma}{V} \\ 0 \end{bmatrix},$$

and the input vector fields take the form:

$$g_1(\bar{\mathbf{x}}) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad g_2(\bar{\mathbf{x}}) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \frac{1}{V} \\ 0 \end{bmatrix}, \quad g_3(\bar{\mathbf{x}}) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \frac{1}{V\cos\gamma} \end{bmatrix}.$$

Taking the system outputs as:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} p_n \\ p_e \\ h \end{bmatrix},$$

the Lie derivatives are computed to analyze the system's control dependencies, construct the decoupling matrix, and determine the system's relative degrees. For clarity and consistency in notation, all gradients are computed with respect to the nonlinear state vector. To simplify the representation, the notation ∇ is adopted in place of $\nabla_{\bar{\mathbf{x}}}$.
The computation of the first-order Lie derivatives proceeds as follows:

$$\begin{aligned} \dot{y}_i = L_f y_i(\bar{\mathbf{x}}) &= \sum_{k=1}^{6} \frac{\partial y_i}{\partial \bar{x}_k} f_k(\bar{\mathbf{x}}) \\ &= \nabla y_i(\bar{\mathbf{x}}) \cdot f(\bar{\mathbf{x}}). \end{aligned}$$

Specifically, for $y_1 = p_n$, the gradient is

$$\nabla y_1 = \nabla p_n = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^T$$

and its first Lie derivative is

$$L_f y_1 = \nabla y_1 \cdot f(\bar{\mathbf{x}}) = V\cos\gamma\cos\chi.$$

Applying the same procedure for y_2 and y_3 leads to

$$L_f y_2 = V\cos\gamma\sin\chi, \quad L_f y_3 = V\sin\gamma.$$

It is essential to highlight that none of these expressions explicitly contain the control inputs a_i, indicating that additional differentiation is required to establish the necessary control dependencies.

Consequently, the computation of the second-order Lie derivatives becomes necessary to determine the explicit dependence of the system's outputs on the control inputs. The second Lie derivative of a given output function y_i is defined as follows:

$$\begin{aligned}\ddot{y}_i = L_f^2 y_i = L_f\big(L_f y_i\big) &= \frac{\partial(L_f y_i)}{\partial \bar{\mathbf{x}}} \cdot f(\bar{\mathbf{x}}) + \sum_{j=1}^{3} L_{g_j}(L_f y_i)\, a_j \\ &= \underbrace{\nabla\big(L_f y_i\big) \cdot f(\bar{\mathbf{x}})}_{\text{drift term}} + \sum_{j=1}^{3} \underbrace{L_{g_j}(L_f y_i)\, a_j}_{\text{input terms}}\end{aligned}$$

where the drift term represents the autonomous contribution of the system's dynamics, while the input terms explicitly incorporate the effects of the control inputs. The second Lie derivatives appearing in the input terms are computed as follows:

$$\begin{aligned}L_{g_j}(L_f y_i) &= \sum_{k=1}^{6} \frac{\partial(L_f y_i)}{\partial \bar{x}_k}\, g_{j,k}(\bar{\mathbf{x}}) \\ &= \nabla\left(L_f y_i\right) \cdot g_j(\bar{\mathbf{x}}).\end{aligned}$$

To illustrate this computation, consider the specific case where the output function is given by $y_1 = p_n$. The drift term is obtained by computing the Lie derivative:

$$\nabla(L_f y_1) \cdot f(\bar{\mathbf{x}}),$$

where the gradient of $L_f y_1$ is determined as:

$$\nabla(L_f y_1) = \begin{bmatrix} 0 & 0 & 0 & \cos\gamma\cos\chi & -V\sin\gamma\cos\chi & -V\cos\gamma\sin\chi \end{bmatrix}^T.$$

Taking the dot product with $f(\bar{\mathbf{x}})$, we obtain:

$$\nabla(L_f y_1) \cdot f(\bar{\mathbf{x}}) = -g\sin\gamma\cos\gamma\cos\chi + g\sin\gamma\cos\gamma\cos\chi = 0.$$

This confirms that the drift term does not contribute to the acceleration along p_n. The input term is then computed as:

$$L_{g_j}(L_f y_1) = \nabla\left(L_f y_1\right) \cdot g_j(\bar{\mathbf{x}}).$$

Since the gradient $\nabla(L_f y_1)$ is identical to that used in the drift term, the components of $L_{g_j}(L_f y_1)$ for each input direction g_j are given by:

$$\begin{aligned} L_{g_1}(L_f y_1) &= \nabla(L_f y_1) \cdot g_1 = \cos\gamma\cos\chi, \\ L_{g_2}(L_f y_1) &= \nabla(L_f y_1) \cdot g_2 = -\sin\gamma\cos\chi, \\ L_{g_3}(L_f y_1) &= \nabla(L_f y_1) \cdot g_3 = -\sin\chi. \end{aligned}$$

The same methodology can be systematically applied to the remaining output components to compute their corresponding second Lie derivatives determining their explicit dependence on the control inputs.
These computed second Lie derivatives can be effectively represented in matrix form as follows:

$$\ddot{\mathbf{y}} = \mathbf{b}(\bar{\mathbf{x}}) + \mathcal{A}(\bar{\mathbf{x}})\mathbf{a}$$

in which the drift term $\mathbf{b}(\bar{\mathbf{x}})$ is given by

$$\mathbf{b}(\bar{\mathbf{x}}) = \begin{bmatrix} L_f(L_f y_1) \\ L_f(L_f y_2) \\ L_f(L_f y_3) \end{bmatrix} = \begin{bmatrix} \nabla(L_f y_1) \cdot f(\bar{\mathbf{x}}) \\ \nabla(L_f y_2) \cdot f(\bar{\mathbf{x}}) \\ \nabla(L_f y_3) \cdot f(\bar{\mathbf{x}}) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix},$$

and the decoupling matrix in the input term is given by

$$\begin{aligned} \mathcal{A}(\bar{\mathbf{x}}) &= \begin{bmatrix} L_{g_1}(L_f y_1) & L_{g_2}(L_f y_1) & L_{g_3}(L_f y_1) \\ L_{g_1}(L_f y_2) & L_{g_2}(L_f y_2) & L_{g_3}(L_f y_2) \\ L_{g_1}(L_f y_3) & L_{g_2}(L_f y_3) & L_{g_3}(L_f y_3) \end{bmatrix} \\ &= \begin{bmatrix} \cos\gamma\cos\chi & -\sin\gamma\cos\chi & -\sin\chi \\ \cos\gamma\sin\chi & -\sin\gamma\sin\chi & \cos\chi \\ \sin\gamma & \cos\gamma & 0 \end{bmatrix}. \end{aligned}$$

It is crucial to ensure that $\mathcal{A}(\mathbf{x})$ remains invertible for the feedback linearization process to be feasible. The determinant of this matrix is given by $\det(\mathcal{A}(\mathbf{x})) = \cos\gamma$, which remains nonzero provided that $\gamma \neq \pm\frac{\pi}{2}$, meaning that the aircraft is not oriented in a purely vertical direction. Consequently, under normal flight conditions, $\mathcal{A}(\mathbf{x})$ is invertible, ensuring the applicability of the feedback linearization technique.
The relative degree r_i of each output y_i corresponds to the number of times it must be differentiated along the system dynamics before an explicit de-

pendence on the control input appears. As previously established, the first Lie derivatives $L_f y_i$ do not involve the control inputs, whereas they appear in the second Lie derivatives $L_f^2 y_i$ through the terms $L_{g_j}(L_f y_i)$. This observation implies that each output y_i possesses a relative degree of $r_i = 2$. Given that there are three outputs, the total relative degree sums to $2+2+2=6$, which matches the dimension of the state vector, confirming that the system is fully feedback linearizable.

Consequently, the linearized system in state-space form, with the redefined state vector $\mathbf{x} = \begin{bmatrix} p_n & p_e & h & \dot{p}_n & \dot{p}_e & \dot{h} \end{bmatrix}^T$, can be expressed as follows:

$$\dot{\mathbf{x}} = \mathbf{F}\mathbf{x} + \mathbf{G}\mathbf{u},$$

where the state-space matrices $\mathbf{F}$ and $\mathbf{G}$ are defined as

$$\mathbf{F} = \begin{bmatrix} \mathbf{0}_{3\times3} & \mathbf{I}_{3\times3} \\ \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3} \end{bmatrix}, \quad \mathbf{G} = \begin{bmatrix} \mathbf{0}_{3\times3} \\ \mathbf{I}_{3\times3} \end{bmatrix}.$$

The transformed control inputs $\mathbf{u}$ are given by

$$\mathbf{u} = \ddot{\mathbf{y}} = \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} + \begin{bmatrix} \cos\gamma\cos\chi & -\sin\gamma\cos\chi & -\sin\chi \\ \cos\gamma\sin\chi & -\sin\gamma\sin\chi & \cos\chi \\ \sin\gamma & \cos\gamma & 0 \end{bmatrix} \mathbf{a},$$

which corresponds to a standard double integrator system in the new state coordinates $\mathbf{x}$ with modified control inputs $\mathbf{u}$. This formulation aligns with the classical representation of fully linearized systems via feedback linearization [11].

1.5 Constraints

The dynamics of fixed-wing aircraft impose fundamental constraints that govern their motion and operational feasibility. These constraints arise from a combination of physical, aerodynamic, and control limitations that dictate the aircraft's behavior. A principal challenge in analyzing the motion of fixed-wing aircraft lies in the intrinsic coupling between velocity, maneuverability, and aerodynamic forces. Unlike multirotor UAVs, which can execute aggressive maneuvers with rapid changes in velocity and direction, fixed-wing aircraft must maintain continuous forward motion, adhere to a minimum turn radius, and regulate altitude transitions while preserving aerodynamic effi-

ciency.
The constraints affecting fixed-wing aircraft can be broadly categorized into kinematic constraints, which describe motion based on the vehicle's geometric and nonholonomic properties, and dynamic constraints, which encompass the forces and moments acting on the system. However, these categories cannot be considered independently, as the dynamics of fixed-wing aircraft introduce complex couplings between aerodynamic forces, control inputs, and resulting kinematics. The equations of motion governing the aircraft inherently link these constraints, requiring a comprehensive understanding of their interactions to accurately describe the vehicle's behavior.
The following discussion provides a structured overview of these constraints, detailing both kinematic and dynamic limitations that must be incorporated into the motion planning framework. By addressing these interdependencies, it becomes possible to generate feasible trajectories that respect the aircraft's physical and aerodynamic constraints while ensuring stable and efficient flight.

1.5.1 Kinematic Constraints

Fixed-wing aircraft are subject to nonholonomic constraints, which fundamentally differentiate their motion from that of rotary-wing UAVs. Unlike multirotor platforms, fixed-wing aircraft are incapable of hovering, lateral movement, or abrupt changes in direction. Their movement is governed by continuous forward motion, necessitating the generation of smooth, feasible trajectories that adhere to these restrictions.
A primary kinematic limitation is the bounded airspeed range, denoted as $V_{\min} \leq V \leq V_{\max}$. The lower bound, $V_{\min}$, is critical for maintaining sufficient lift and preventing aerodynamic stall, whereas the upper bound, $V_{\max}$, is constrained by structural and aerodynamic considerations. Any deviation beyond this range compromises flight stability and control authority, making compliance with these limits essential for safe and efficient operation.
Furthermore, the aircraft's maneuverability is constrained by the physical limitations imposed by its aerodynamic design and actuation capabilities. One of the most significant constraints is the maximum allowable bank angle, $\phi_{\max}$, which directly influences both the minimum turning radius and the turn rate.
The minimum turning radius is dictated by the aircraft's velocity and banking capability. Since the vehicle is unable to rotate in place or execute instantaneous turns, it must follow continuous trajectories that adhere to these

constraints. The minimum turning radius is derived from the relationship between the aircraft's speed and its achievable bank angle, which renders sharp turns infeasible at high velocities without exceeding structural or aerodynamic limits. By summing the vertical force components shown in Figure 13, the vertical component of the lift force is expressed as equal and opposite to the projection of the gravitational force onto the $\mathbf{j}^b - \mathbf{k}^b$ plane:

$$F_{\text{lift}} \cos\phi = mg \cos\gamma$$

By dividing this expression by equation (2) and solving for $\dot{\chi}$, the following result is obtained:

$$\dot{\chi} = \frac{g \tan\phi}{V_g} \tag{4}$$

which describes a coordinated turn. Given that the turning radius is defined as

$$R = \frac{V_g \cos\gamma}{\dot{\chi}},$$

substituting equation (4) into this expression, under the assumptions of constant level flight ($\cos\gamma = 1$) and negligible wind effects ($V_g \approx V$), the minimum turning radius is given by

$$R_{\min} = \frac{V^2}{g \tan\phi_{\max}}.$$

Additionally, the relationship between the bank angle and the turn rate, recalling the approximation $\dot{\chi} \approx \dot{\psi}$, is given by

$$\dot{\psi} = \frac{g \tan\phi}{V}. \tag{5}$$

This equation, which is widely found in the literature, highlights the fact that higher turn rates necessitate greater bank angles, which are physically limited by $\phi_{\max}$.

In addition to banking limitations, the aircraft is also subject to constraints on pitch angles. Excessive pitch values can induce aerodynamic stall, wherein the airflow over the wings separates, leading to a sudden loss of lift. Consequently, all planned trajectories must conform to these kinematic constraints to ensure the safe and efficient operation of the fixed-wing aircraft.

1.5.2 Dynamic Constraints

Beyond purely kinematic limitations, the aircraft's motion is governed by its dynamic characteristics and actuator capabilities. The forces acting on the vehicle—including lift, drag, thrust, and gravity—must remain within feasible limits to ensure stable flight.

One of the primary dynamic constraints is the limitation on control inputs due to actuator saturation. Control surfaces, including ailerons, elevators, and rudders, are subject to mechanical and aerodynamic restrictions that limit their deflection ranges, thereby constraining the achievable angular accelerations. These limitations have a direct impact on the aircraft's response time and maneuverability, particularly in scenarios requiring rapid turns or altitude transitions.

Another critical constraint arises from the limitations of the propulsion system. The available thrust is bounded by a maximum thrust value, which dictates the upper limit on acceleration and climb performance. This constraint is particularly relevant in high-speed maneuvers and steep ascents, where insufficient thrust may prevent the aircraft from maintaining altitude or accelerating as required.

Additionally, aerodynamic force constraints govern the feasible climb and descent rates. As previously discussed, the aircraft's ability to ascend or descend is dependent on the interplay between lift, weight, and available thrust. Excessive climb rates may lead to aerodynamic stall, while uncontrolled descents risk exceeding structural load limits. Consequently, all planned trajectories must be designed to respect these constraints, ensuring safe operation under varying flight conditions.

Finally, the maximum allowable climb and descent rates impose further limitations on motion planning. Steep climbs necessitate increased thrust and sufficient airflow over the control surfaces to maintain stability, whereas rapid descents may lead to excessive airspeed buildup or structural stress. These constraints require planned trajectories to incorporate gradual altitude transitions, avoiding abrupt changes that could compromise the aircraft's structural integrity or controllability.

2 Path Planning Algorithm

This chapter provides a comprehensive overview of the Rapidly-exploring Random Tree algorithm [5, 6], along with its enhanced variant, Rapidly-exploring Random Tree Star [6, 12], and subsequently introduces the Kinodynamic RRT* algorithm [10], which is employed in this work to achieve the specified objective. Additionally, the principles of optimal control that sustain the algorithm are examined, drawing from the theoretical foundations established in [13].
Following this, the methodology for environment modeling is presented, including the formal definition of the costmap utilized in the planning framework, along with the constraints imposed on it. This discussion provides the necessary context for understanding how the environment is represented and integrated into the path planning process.
Lastly, key implementation details are outlined with the aim of simplifying and optimizing computational efficiency. These include techniques such as polynomial root-finding through the exploitation of the companion matrix form, leveraging the nilpotency property of the system as derived in Section 1.4, and an analysis of tree search algorithms based on [14].

2.1 Rapidly-exploring Random Tree

The Rapidly-exploring Random Tree (RRT) algorithm is a widely utilized, incremental, sampling-based motion planning technique in robotics and autonomous systems, designed to efficiently explore complex and high-dimensional spaces in an attempt to find a path from a start configuration to a goal configuration. RRT excels in quickly generating an initial solution in complex or uncertain environments, making it a favorite for applications where finding any feasible path rapidly is more critical than guaranteeing its quality.
The core of the RRT algorithm involves constructing a tree of feasible, collision-free trajectories. This tree grows incrementally, starting from an initial configuration, and is expanded towards randomly sampled points in the configuration space. The algorithm continues to extend the tree until either a viable path to a neighborhood of the goal is found, or a termination criterion—such as a specified number of iterations or a time limit—has been reached. The iterative process of exploring and expanding the tree allows the algorithm to quickly explore large portions of the configuration space, making it well-suited for applications involving robotic manipulation or autonomous vehicles, where

rapid decision-making is crucial.

An illustrative example of an RRT-generated path within a two-dimensional environment is shown in Figure 14, providing a visual representation of how the tree grows over time towards the goal configuration. The tree structure is composed of nodes, each representing a valid configuration within the state space, and edges, which correspond to collision-free trajectories connecting these configurations. The tree starts with a root node that represents the initial state and gradually expands towards the goal configuration, adding new nodes and edges as feasible paths are identified.

At each iteration of the algorithm, a random configuration is sampled from the free space of the configuration space. This random sample is generated without any additional constraints, which facilitates the unbiased exploration of the space. The next step is to find the nearest node in the tree to the randomly sampled configuration, typically using a Euclidean distance metric. Once the nearest node is identified, the algorithm applies a steering function that directs the tree from the current node towards the random sample. This steering function is essential for determining how the tree should extend towards the new configuration and, in its simplest form, is typically implemented through a straight-line trajectory between the two nodes.

Once the new candidate configuration is generated, it is subject to a collision check to ensure that the trajectory connecting the current node and the new configuration does not intersect any obstacles in the environment. If the path is found to be collision-free, the new configuration is added to the tree as a child node of the current node. The algorithm terminates successfully when the newly added configuration lies within a predefined goal region. This process ensures that the algorithm is not only searching for a path but is also taking into account the constraints imposed by obstacles in the environment.

The principal strengths of the RRT algorithm lie in its computational efficiency and rapid convergence properties. By exploring the configuration space in an incremental and random manner, RRT is able to navigate through large, high-dimensional spaces that would be computationally intractable for traditional methods. Its performance in environments with obstacles is particularly notable, as the tree grows in a manner that avoids collisions, ensuring that only valid paths are explored.

However, while the RRT algorithm excels at finding initial solutions quickly, it has some notable limitations. One of the most significant drawbacks is the lack of asymptotic optimality. Although the algorithm is capable of find-

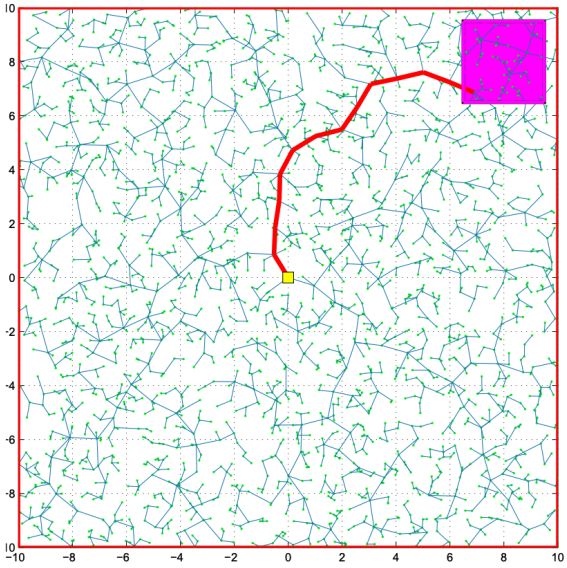

Figure 14: Illustration of RRT path planning in a 2D free environment. The tree structure (light blue) explores the space with random nodes (green) until finding a path (red) from start to a goal region (purple) [6].

ing a feasible path quickly, it does not guarantee optimality, as it prioritizes rapidly exploring the configuration space over optimizing the path length or smoothness. Repeated runs of the algorithm can yield different solutions, some of which may be significantly suboptimal. As a result, the paths found by RRT can be jagged or inefficient, and further post-processing, such as smoothing or shortcutting, is often required to improve their quality. Additionally, the stochastic nature of the algorithm means that in environments with narrow passages or dense obstacles, RRT may fail to find a solution or take an excessive amount of time to do so. The algorithm's performance is highly dependent on the distribution of the random samples, and in regions with sparse sampling, the growth of the tree may be slow or stagnant.

2.2 Rapidly-exploring Random Tree Star

The Rapidly-exploring Random Tree Star (RRT*) algorithm is an enhanced version of the standard RRT algorithm, offering significant improvements in path quality by incorporating optimality into the motion planning process. Unlike traditional RRT, which prioritizes rapid exploration of the configuration space without consideration for path optimality, RRT* aims to refine the generated paths, gradually improving them as the tree expands. This modification allows RRT* to not only explore the configuration space efficiently but also to converge towards the optimal path as the number of iterations

increases, ensuring that the solution becomes more optimal over time.

RRT* retains the core principles of RRT, including the incremental construction of a tree from an initial configuration and the sampling of random configurations in the free space. However, RRT* distinguishes itself by introducing a new mechanism to evaluate and improve the quality of paths as the tree grows. In particular, RRT* utilizes a rewiring process, where new nodes are not simply added to the tree but are used to potentially optimize the connections between existing nodes. This rewiring step ensures that the path from the start configuration to the goal configuration is continually refined during the algorithm's execution, as shown in Figure 15.

At each step, after a new node is added to the tree, the algorithm checks the neighboring nodes and potentially rewires the tree by replacing less optimal connections with more efficient ones. This iterative improvement process allows the algorithm to gradually converge to the shortest possible path, provided enough time is allocated. The rewiring step is critical because it balances the need for efficient exploration with the desire for an optimal solution, ensuring that the resulting path is not just feasible but also optimal in terms of length. By gradually refining the path during the search process, RRT* guarantees that, with sufficient computational resources and time, it will converge to an optimal path. This property is often referred to as asymptotic optimality, meaning that the algorithm's performance improves as the number of iterations increases, and the path found approaches the true optimal solution as the number of samples grows.

In terms of advantages, RRT* introduces a significant enhancement over standard RRT by improving the overall path quality while retaining the ability to efficiently explore high-dimensional spaces. The algorithm is particularly useful in environments with obstacles, where its ability to find a feasible path while minimizing the length of the trajectory is essential. The rewiring mechanism contributes to a significant reduction in path length compared to RRT, making it an ideal choice for applications requiring both collision avoidance and path optimization.

However, despite its advantages, RRT* is not without its limitations. One of the primary drawbacks of RRT* is its computational complexity. The rewiring process, while beneficial in terms of path quality, introduces additional computational overhead. This can result in longer execution times compared to RRT, particularly in large or highly constrained configuration spaces. The increased computation is due to the need to evaluate multiple

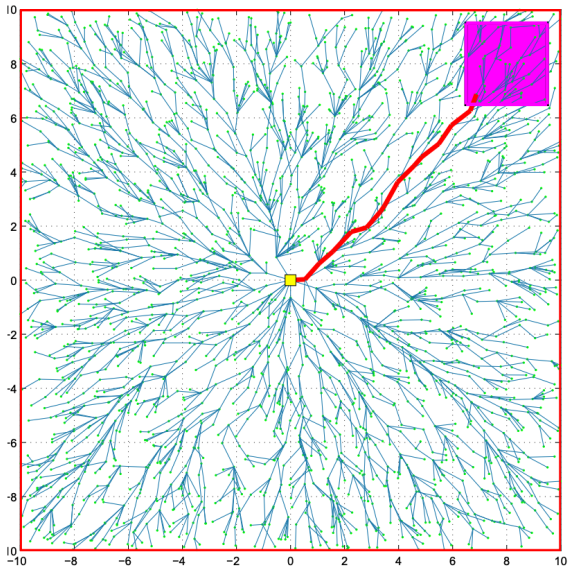

Figure 15: Illustration of RRT* path planning in a 2D free environment. The tree structure (light blue) explores the space with random nodes (green) until finding a path (red) from start to a goal region (purple) [6].

potential connections for each new node, which can be time-consuming, especially in high-dimensional spaces where the number of possible connections grows exponentially.

Another limitation of RRT* is its sensitivity to the density and distribution of sampled points. While the algorithm is probabilistically complete, meaning it will eventually find a solution if one exists, the quality of the solution can be influenced by the sampling strategy. In environments with narrow passages or regions of low connectivity, the random sampling method may fail to adequately explore the space, leading to suboptimal paths or even failure to find a solution in some cases. Furthermore, the algorithm's performance is highly dependent on the number of samples taken, meaning that achieving asymptotic optimality may require a large number of iterations, which can be impractical in online applications or environments with time constraints.

Additionally, RRT* does not guarantee smoothness of the resulting path. While the path length is optimized, the trajectory between nodes may still exhibit sharp turns or other non-smooth features, which may require further refinement or post-processing, such as trajectory smoothing algorithms, to improve the path's quality for practical use.

2.3 Kinodynamic Rapidly-exploring Random Tree Star

The Kinodynamic Rapidly-exploring Random Tree Star (Kinodynamic RRT*) algorithm extends the capabilities of the standard RRT* by incorporating both dynamic constraints and kinodynamic optimization into the motion planning process. Unlike traditional motion planning algorithms, which primarily focus on finding feasible paths within a configuration space, Kinodynamic RRT* addresses both the kinematic and dynamic properties of the system, such as velocity and acceleration constraints, ensuring that the generated paths are not only collision-free but also physically feasible. This makes Kinodynamic RRT* particularly valuable for real-world applications, where the motion of a robotic system is constrained by its dynamics, such as in autonomous vehicle navigation, robotic arm manipulation, or aerial vehicle flight planning.

The primary distinction of Kinodynamic RRT* from standard RRT* lies in its treatment of the system's dynamics. While RRT* focuses on minimizing path length and refining the trajectory based on geometrical constraints, Kinodynamic RRT* accounts for the system's velocity, acceleration, and other dynamic parameters throughout the planning process. This enables the algorithm to generate paths that are feasible not only in the static sense but also in terms of the system's motion capabilities. The inclusion of dynamic constraints allows Kinodynamic RRT* to produce trajectories that respect the system's physical limitations, ensuring smooth, realistic motion that can be executed by the robot or vehicle.

The algorithm works by incrementally constructing a tree, much like RRT*, but with additional considerations for the system's dynamics. At each iteration, Kinodynamic RRT* samples a random configuration and attempts to connect the tree to it using an optimal control law that incorporates both the kinematic and dynamic properties of the system, such as velocity and acceleration profiles. Once a new node is added to the tree, the algorithm performs a rewiring process to optimize the tree, ensuring that the path from the start to the goal configuration is not only feasible but also optimal in terms of the defined cost function.

The integration of dynamic constraints is achieved through the use of control inputs that respect the system's dynamics, such as acceleration limits, maximum velocities, and other system-specific parameters. This makes the Kinodynamic RRT* algorithm highly suitable for systems that must respect these constraints, such as in autonomous vehicles required to adhere to veloc-

ity and acceleration limits and other dynamic restrictions. Furthermore, by incorporating these constraints into the optimization process, Kinodynamic RRT* ensures that the final path is not just a feasible trajectory but also one that can be physically realized by the system.

One of the major advantages of Kinodynamic RRT* over standard RRT* is its ability to generate paths that are both optimal and physically feasible. By taking into account the system's dynamics, the algorithm ensures that the generated paths can be executed by real-world systems without violating their physical constraints. This is particularly important in applications where the robot or vehicle must operate within a physically constrained environment, such as when avoiding obstacles while maintaining a specific speed or acceleration profile. Additionally, Kinodynamic RRT* retains the optimality guarantees of RRT*, ensuring that the generated path is not only feasible but also converges to an optimal solution as the number of iterations increases.

Another key advantage of Kinodynamic RRT* is its ability to handle high-dimensional, highly constrained configuration spaces. Like RRT* and RRT, Kinodynamic RRT* is particularly effective in environments with complex obstacles or narrow passages, where traditional motion planning methods struggle to find feasible solutions. The incorporation of dynamic constraints ensures that the algorithm can efficiently explore such environments while maintaining the physical feasibility of the generated paths.

Despite its advantages, Kinodynamic RRT* also has several limitations. One of the primary drawbacks is its increased computational complexity compared to standard RRT* and RRT. The inclusion of dynamic constraints means that each connection between two states must consider not only the geometric properties of the configuration space but also the system's dynamic properties, which can significantly increase the computational cost. Additionally, the rewiring process, essential for optimizing the path, becomes more complex when dynamic constraints are involved, further contributing to the algorithm's computational burden.

Furthermore, the Kinodynamic RRT* algorithm is sensitive to the accuracy of the system's dynamic model. If the dynamic model is inaccurate or incomplete, the algorithm may generate paths that are feasible in theory but cannot be executed in practice due to model discrepancies. This issue can be mitigated by using more accurate models, but this comes at the cost of increased computational complexity.

2.3.1 Problem Formulation

Let $\mathcal{X} = \mathbb{R}^n$ and $\mathcal{U} = \mathbb{R}^m$ be the state space and control input space, respectively, and let the dynamics be defined by the following linear time invariant system:

$$\dot{\mathbf{x}}(t) = \mathbf{F}\mathbf{x}(t) + \mathbf{G}\mathbf{u}(t) \tag{6}$$

where $\mathbf{x}(t) \in \mathcal{X}$ is the state, $\mathbf{u}(t) \in \mathcal{U}$ is the control input, $\mathbf{F} \in \mathbb{R}^{n\times n}$ and $\mathbf{G} \in \mathbb{R}^{m\times m}$.

A trajectory is defined by a tuple

$$\pi = (\mathbf{x}(\cdot), \mathbf{u}(\cdot), \tau),$$

where τ is the arrival time or duration of the trajectory, $\mathbf{u} : [0, \tau] \to \mathcal{U}$ defines the control input along the trajectory, and $\mathbf{x} : [0, \tau] \to \mathcal{X}$ are the corresponding states along the trajectory given $\mathbf{x}(0)$ with linear dynamic (6). The cost of a trajectory π is defined by the following functional:

$$J(\pi) = \int_0^\tau \left(1 + \mathbf{u}(t)^T \mathbf{R}\mathbf{u}(t)\right) dt, \tag{7}$$

which penalizes both the duration of the trajectory and the control effort, where $\mathbf{R} = \mathbf{R}^T \succ 0$ is positive-definite, constant, and weights the cost of the control inputs.

Let $\mathcal{X}_{\text{free}} \subset \mathcal{X}$ define the free state space, which consists of those states that are within user-defined bounds and are collision-free with respect to obstacles in the environment. Similarly, let $\mathcal{U}_{\text{free}} \subset \mathcal{U}$ define the free control input space, consisting of control inputs that are within bounds placed on them.

The problem under consideration is to determine an optimal trajectory for a system, given a start state $\mathbf{x}_{\text{start}} \in \mathcal{X}_{\text{free}}$ and a goal state $\mathbf{x}_{\text{goal}} \in \mathcal{X}_{\text{free}}$, where the objective is to find a collision-free path π^*_{free} that connects these two states while minimizing the associated cost. Specifically, the problem is formulated as follows:

$$\begin{aligned} \pi^*_{\text{free}} = \underset{\pi}{\text{argmin}} \quad & J(\pi) \\ \text{s.t.} \quad & \mathbf{x}(0) = \mathbf{x}_{\text{start}} \\ & \mathbf{x}(\tau) = \mathbf{x}_{\text{goal}} \\ & \mathbf{x}(t) \in \mathcal{X}_{\text{free}}, \quad \forall t \in [0, \tau] \\ & \mathbf{u}(t) \in \mathcal{U}_{\text{free}}, \quad \forall t \in [0, \tau] \end{aligned} \tag{8}$$

where $J(\pi)$ represents the cost function that penalizes both the trajectory

duration and the control effort. The states $\mathbf{x}(t)$ are constrained to remain within the free space $\mathcal{X}_{\text{free}}$, and the control inputs $\mathbf{u}(t)$ must lie within the feasible control set $\mathcal{U}_{\text{free}}$. The solution to this problem requires computing the trajectory that satisfies the given boundary conditions and minimizes the cost function.

The key to solving this problem lies in the ability to compute the optimal trajectory between any two states $\mathbf{x}_i$ and $\mathbf{x}_j$, as well as its associated optimal cost. For any pair of states $\mathbf{x}_i, \mathbf{x}_j \in \mathcal{X}_{\text{free}}$, the optimal trajectory is determined by solving the following problem:

$$\begin{aligned} \pi^*(\mathbf{x}_i, \mathbf{x}_j) = \underset{\pi}{\text{argmin}} \quad & J(\pi) \\ \text{s.t.} \quad & \mathbf{x}(0) = \mathbf{x}_i \\ & \mathbf{x}(\tau) = \mathbf{x}_j \end{aligned}$$

The optimal cost associated with this trajectory is given by:

$$\begin{aligned} J^*(\mathbf{x}_i, \mathbf{x}_j) = \min_{\pi} \quad & J(\pi) \\ \text{s.t.} \quad & \mathbf{x}(0) = \mathbf{x}_i \\ & \mathbf{x}(\tau) = \mathbf{x}_j \end{aligned} \tag{9}$$

Importantly, it is observed that the optimal cost can be decomposed into the sum of the costs of sub-trajectories between successive intermediate states. Specifically:

$$J^*(\mathbf{x}_i, \mathbf{x}_j) = J^*(\mathbf{x}_i, \mathbf{x}(t)) + J^*(\mathbf{x}(t), \mathbf{x}_j) \quad 0 < t < \tau$$

Thus, an optimal collision-free trajectory, denoted as π^*_{free}, between the start state $\mathbf{x}_{\text{start}}$ and the goal state $\mathbf{x}_{\text{goal}}$, is composed of a sequence of optimal sub-trajectories that connect a series of successive states $(\mathbf{x}_{\text{start}}, \mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_{\text{goal}})$ within the free state space $\mathcal{X}_{\text{free}}$.

This problem can initially be framed as an optimal control problem with fixed final states and fixed final time. However, a transformation is possible, wherein the final time is allowed to vary freely, while the final state remains fixed. This transformation is critical, as it enables the computation of both the optimal trajectory and its corresponding cost function. The next section outlines the process of solving this optimal control problem under these transformed conditions, detailing how the optimal trajectory can be computed, and the associated cost function minimized, given the specified constraints.

2.3.2 Optimal Control Law

The problem at hand involves determining an optimal trajectory $(\mathbf{x}(\cdot), \mathbf{u}(\cdot), \tau)$ that satisfies a set of constraints and minimizes a given cost function. Specifically, given a fixed final time τ and two states $\mathbf{x}_i$ and $\mathbf{x}_j$, the objective is to find a control trajectory that minimizes the cost function while adhering to the system's dynamics and boundary conditions. The optimal control problem is formulated as:

$$\begin{aligned}(\mathbf{x}(\cdot), \mathbf{u}(\cdot), \tau) = \underset{\pi}{\text{argmin}} \quad & J(\pi) \\ \text{s. t.} \quad & \mathbf{x}(0) = \mathbf{x}_i \\ & \mathbf{x}(\tau) = \mathbf{x}_j \\ & \dot{\mathbf{x}}(t) = \mathbf{F}\mathbf{x}(t) + \mathbf{G}\mathbf{u}(t), \quad 0 < t \leq \tau\end{aligned}$$

This represents the fixed final state, fixed final time optimal control problem, wherein the trajectory must satisfy the state dynamics and boundary conditions. The state must evolve according to the system dynamics in (6), and at the end of the trajectory, the state $\mathbf{x}(\tau)$ must match the goal state $\mathbf{x}_j$. The optimal control solution involves finding the trajectory $\mathbf{x}(t)$ and control input $\mathbf{u}(t)$ that minimize the cost functional $J(\pi)$ while satisfying these constraints.

To solve this problem, the Pontryagin Minimum Principle (PMP) is employed, which necessitates the introduction of the Hamiltonian for the system. The Hamiltonian is given by:

$$\mathcal{H}(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}, t) = \underbrace{1 + \mathbf{u}^T\mathbf{R}\mathbf{u}}_{\text{Running cost}} + \underbrace{\boldsymbol{\lambda}^T(\mathbf{F}\mathbf{x} + \mathbf{G}\mathbf{u})}_{\text{Dynamics adjoint}},$$

where $\boldsymbol{\lambda}(t)$ denotes the costate vector. This Hamiltonian reflects both the running cost associated with the control input and the dynamics of the system. The next step involves solving for the costate, which evolves according to the costate dynamic derived from the Hamiltonian:

$$\dot{\boldsymbol{\lambda}}(t) = -\frac{\partial \mathcal{H}}{\partial \mathbf{x}} = -\mathbf{F}^T\boldsymbol{\lambda}(t), \tag{10}$$

which is a backward differential equation that describes the evolution of the costate vector. The solution of (10) is the costate equation given by:

$$\boldsymbol{\lambda}(t) = e^{\mathbf{F}^T(\tau - t)}\boldsymbol{\lambda}(\tau) \tag{11}$$

where $\boldsymbol{\lambda}(\tau)$ represents the terminal costate at the final time τ.
The optimal control law is derived by minimizing the Hamiltonian with respect to the control input $\mathbf{u}(t)$as follows:

$$\frac{\partial \mathcal{H}}{\partial \mathbf{u}} = 2\mathbf{R}\mathbf{u} + \mathbf{G}^T\boldsymbol{\lambda} = 0$$

which leads to the following optimal control:

$$\mathbf{u}(t) = -\frac{1}{2}\mathbf{R}^{-1}\mathbf{G}^T\boldsymbol{\lambda}(t). \tag{12}$$

At this point, it is useful to introduce a new state trajectory $\bar{\mathbf{x}}(t)$, which represents the evolution of the state $\mathbf{x}(t)$ when no control input is applied (i.e., $\mathbf{u}(t) = 0$). This trajectory evolves according to:

$$\bar{\mathbf{x}}(t) = e^{\mathbf{F}t}\mathbf{x}_i \tag{13}$$

The actual state trajectory under optimal control is then given by:

$$\mathbf{x}(t) = \bar{\mathbf{x}}(t) + \int_0^t e^{\mathbf{F}(t-s)}\mathbf{G}\mathbf{u}(s)\, ds.$$

At the final time $t = \tau$, the state must satisfy the boundary condition $\mathbf{x}(\tau) = \mathbf{x}_j$, which leads to:

$$\mathbf{x}_j = \bar{\mathbf{x}}(\tau) + \int_0^\tau e^{\mathbf{F}(\tau-s)}\mathbf{G}\mathbf{u}(s)\, ds.$$

Substituting the expression for the optimal control (12) and using the costate solution (11), the state constraint becomes:

$$\mathbf{x}_j - \bar{\mathbf{x}}(\tau) = -\frac{1}{2}\int_0^\tau e^{\mathbf{F}(\tau-s)}\mathbf{G}\mathbf{R}^{-1}\mathbf{G}^T e^{\mathbf{F}^T(\tau-s)}\, ds \cdot \boldsymbol{\lambda}(\tau). \tag{14}$$

To simplify this expression, the weighted reachability Gramian $\mathcal{W}_t$ is introduced, which is defined as:

$$\mathcal{W}_t = \int_0^t e^{\mathbf{F}(t-t')}\mathbf{G}\mathbf{R}^{-1}\mathbf{G}^T e^{\mathbf{F}^T(t-t')}\, dt' \tag{15}$$

that satisfies the Lyapunov equation:

$$\dot{\mathcal{W}}_t = \mathbf{F}\mathcal{W}_t + \mathcal{W}_t\mathbf{F}^T + \mathbf{G}\mathbf{R}^{-1}\mathbf{G}^T, \quad \mathcal{W}_0 = \mathbf{0}. \tag{16}$$

It is noted that $\mathcal{W}_t$ is a positive-definite matrix and full rank, i.e., $\mathcal{W}_t \succ 0$ and

rank $\mathcal{W}_t = n$ for $t > 0$, iff the system is reachable.
Recognizing the weighted reachability Gramian (15), the state constraint (14) simplifies to:

$$\mathbf{x}_j - \bar{\mathbf{x}}(\tau) = -\frac{1}{2}\mathcal{W}_\tau \boldsymbol{\lambda}(\tau).$$

From this, the terminal costate is derived as:

$$\boldsymbol{\lambda}(\tau) = -2\mathcal{W}_\tau^{-1}(\mathbf{x}_j - \bar{\mathbf{x}}(\tau)). \tag{17}$$

Substituting this terminal costate (17) into the costate equation (11), the costate at any time t is expressed as:

$$\boldsymbol{\lambda}(t) = -2e^{-\mathbf{F}^T(t-\tau)}\mathcal{W}_\tau^{-1}\left(\mathbf{x}_j - \bar{\mathbf{x}}(\tau)\right).$$

To eliminate the factor of -2, a new costate equation $\mathbf{y}(t)$ is introduced, defined as:

$$\mathbf{y}(t) \triangleq -\frac{1}{2}\boldsymbol{\lambda}(t) = e^{-\mathbf{F}^T(t-\tau)}\mathcal{W}_\tau^{-1}\left(\mathbf{x}_j - \bar{\mathbf{x}}(\tau)\right), \tag{18}$$

with the terminal condition:

$$\mathbf{d}(\tau) \triangleq \mathcal{W}_\tau^{-1}(\mathbf{x}_j - \bar{\mathbf{x}}(\tau)) \tag{19}$$

but equal costate dynamic:

$$\dot{\mathbf{y}}(t) = -\mathbf{F}^T\mathbf{y}(t). \tag{20}$$

Finally, in terms of the costate equation (18), the optimal control law is:

$$\mathbf{u}(t) = \mathbf{R}^{-1}\mathbf{G}^T\mathbf{y}(t) \tag{21}$$

which is an open-loop control policy.

2.3.3 Optimal Arrival Time

In order to determine an optimal trajectory $\pi^*(\mathbf{x}_i, \mathbf{x}_j)$ between the initial state $\mathbf{x}_i$ and the final state $\mathbf{x}_j$, as formally defined in (2.3.1), the preceding analysis is extended to address the fixed final state, free final time optimal control problem. This formulation allows the arrival time τ to be chosen freely in a manner that minimizes the cost functional specified in (7). Unlike the fixed-time scenario, where τ is predetermined, this approach introduces an additional degree of freedom, enabling an optimal selection of the arrival time

to achieve a more efficient trajectory in terms of cost minimization.

The determination of the optimal arrival time τ^* follows a structured procedure. By substituting the optimal control policy (21) into the cost functional (7) and subsequently evaluating the integral, a closed-form expression for the total cost associated with an optimal trajectory between $\mathbf{x}_i$ and $\mathbf{x}_j$ is obtained for any fixed arrival time τ:

$$J(\tau) = \tau + \left(\mathbf{x}_j - \bar{\mathbf{x}}(\tau)\right)^T \mathcal{W}_\tau^{-1} \left(\mathbf{x}_j - \bar{\mathbf{x}}(\tau)\right).$$

It is important to note that the cost function $J(\tau)$ is strictly greater than τ for all $\tau > 0$, due to the fact that the weighted reachability Gramian $\mathcal{W}_\tau$ is a positive-definite matrix. This property ensures that the additional term in $J(\tau)$ remains strictly positive, thereby reinforcing the necessity of optimizing τ to minimize the total trajectory cost.

The optimal arrival time τ^* is formally defined as the value of τ that minimizes the cost functional $J(\tau)$, expressed as follows:

$$\tau^* = \underset{\tau>0}{\operatorname{argmin}}\ J(\tau) \tag{22}$$

Once τ^* is identified, the corresponding minimal cost associated with the optimal trajectory between $\mathbf{x}_i$ and $\mathbf{x}_j$, as formulated in (9), is given by:

$$J^*(\mathbf{x}_i, \mathbf{x}_j) = J(\tau^*).$$

In order to determine the precise value of τ^*, the derivative of $J(\tau)$ with respect to τ, denoted by $\dot{J}(\tau)$, is computed, and the optimal arrival time is obtained by solving the condition:

$$\dot{J}(\tau) = 0.$$

By leveraging the Lyapunov equation (16), the derivative of the cost function takes the following explicit form:

$$\dot{J}(\tau) = 1 - 2\left(\mathbf{F}\mathbf{x}_j\right)^T \mathbf{d}(\tau) - \mathbf{d}(\tau)^T \mathbf{G}\mathbf{R}^{-1}\mathbf{G}^T\mathbf{d}(\tau).$$

It is noteworthy that $\dot{J}(\tau)$ is a scalar function and, depending on the system dynamics and boundary conditions, may possess multiple local minima.

2.3.4 Optimal Trajectory

Having determined the optimal arrival time τ^* as specified in (22), it is now possible to derive the corresponding optimal trajectory $\pi^*(\mathbf{x}_i, \mathbf{x}_j)$. This is achieved by formulating the costate equation as given by Equation (18), which incorporates the optimal arrival time into its definition:

$$\mathbf{y}(t) = e^{-\mathbf{F}^T(t-\tau^*)}\mathbf{d}(\tau^*). \tag{23}$$

By substituting the optimal control policy from Equation (21) into the system dynamics described by Equation (6), the differential equation governing the evolution of the state is obtained:

$$\dot{\mathbf{x}}(t) = \mathbf{F}\mathbf{x}(t) + \mathbf{G}\mathbf{R}^{-1}\mathbf{G}^T\mathbf{y}(t), \quad \mathbf{x}(\tau^*) = \mathbf{x}_j. \tag{24}$$

It is important to observe that costate equation for optimal arrival time (23) represents the solution to the costate dynamics expressed in (20), with an initial condition given by:

$$\mathbf{y}(\tau^*) = \mathbf{d}(\tau^*).$$

By combining this result with the state evolution described in (24), the following composite differential equation governing both the state and costate dynamics is derived:

$$\begin{bmatrix}\dot{\mathbf{x}}(t)\\ \dot{\mathbf{y}}(t)\end{bmatrix} = \begin{bmatrix}\mathbf{F} & \mathbf{G}\mathbf{R}^{-1}\mathbf{G}^T\\ 0 & -\mathbf{F}^T\end{bmatrix}\begin{bmatrix}\mathbf{x}(t)\\ \mathbf{y}(t)\end{bmatrix}, \quad \begin{bmatrix}\mathbf{x}(\tau^*)\\ \mathbf{y}(\tau^*)\end{bmatrix} = \begin{bmatrix}\mathbf{x}_j\\ \mathbf{d}(\tau^*)\end{bmatrix}.$$

The general solution to this system is given by:

$$\begin{bmatrix}\mathbf{x}(t)\\ \mathbf{y}(t)\end{bmatrix} = e^{\left(\begin{bmatrix}\mathbf{F} & \mathbf{G}\mathbf{R}^{-1}\mathbf{G}^T\\ 0 & -\mathbf{F}^T\end{bmatrix}(t-\tau^*)\right)}\begin{bmatrix}\mathbf{x}_j\\ \mathbf{d}(\tau^*)\end{bmatrix}.$$

By making use of the optimal control law (21), explicit expressions for both the state $\mathbf{x}(t)$ and the control input $\mathbf{u}(t)$ can be obtained for the entire time interval $0 < t < \tau^*$. These expressions fully characterize the optimal trajectory $\pi^*(\mathbf{x}_i, \mathbf{x}_j)$. With a slight abuse of notation, the optimal state and control input

can be concisely written as:

$$\begin{aligned}\mathbf{x}(t) &= e^{\left(\begin{bmatrix}\mathbf{F} & \mathbf{G}\mathbf{R}^{-1}\mathbf{G}^T\end{bmatrix}(t-\tau^*)\right)}\mathbf{x}_j,\\ \mathbf{u}(t) &= \mathbf{R}^{-1}\mathbf{G}^T e^{\left(\begin{bmatrix}\mathbf{0} & -\mathbf{F}^T\end{bmatrix}(t-\tau^*)\right)}\mathbf{d}(\tau^*).\end{aligned} \tag{25}$$

These formulations provide a complete analytical representation of the optimal state and control trajectories between the initial state $\mathbf{x}_i$ and the final state $\mathbf{x}_j$.

2.3.5 Algorithm

To determine the optimal collision-free trajectory π^*_{free}, as defined in (8), given the capability to compute an optimal trajectory between any pair of states, the kinodynamic RRT* algorithm is utilized. As discussed in Section 2.3 the algorithm is recognized for its asymptotic optimality, implying that as the number of iterations tends to infinity, the probability of identifying an optimal path converges to one. This property guarantees that, provided sufficient computational resources, the algorithm will ultimately yield a solution that minimizes the cost function while ensuring compliance with all imposed constraints.

The algorithm incrementally constructs a tree $\mathcal{T}$ consisting of feasible trajectories within the free state space, where the root node corresponds to the initial state $\mathbf{x}_{\text{start}}$. In each iteration i, a state $\mathbf{x}_i$ is randomly sampled from the free state space $\mathcal{X}_{\text{free}}$ and evaluated as a potential new node for inclusion in the tree. For each newly sampled node, a parent node is selected from the existing nodes in the tree based on proximity and feasibility criteria. Specifically, a neighboring node $\mathbf{x}$ is identified if the cost-to-go function satisfies the condition $J^*(\mathbf{x}, \mathbf{x}_i) < r$, where r represents the neighborhood radius. Among the candidate neighbors, the node $\mathbf{x}$ that results in the minimum-cost, collision-free trajectory $\pi(\mathbf{x}, \mathbf{x}_i)$ is designated as the parent $\mathbf{x}_{near}$. The feasibility of this trajectory is verified by ensuring that both the states and control inputs along the path remain within their respective free spaces.

Upon successfully adding a new node to the tree, the algorithm performs a rewiring procedure aimed at enhancing the overall solution quality. This refinement process involves evaluating all neighboring nodes $\mathbf{x}$ for which $J^*(\mathbf{x}_i, \mathbf{x}) < r$. If a lower-cost, collision-free trajectory from $\mathbf{x}_i$ to $\mathbf{x}$ is found, the parent of $\mathbf{x}$ is reassigned to $\mathbf{x}_i$, thereby reducing the total cost required to reach $\mathbf{x}$ from

the initial state. Through repeated application of this rewiring step, the tree structure is iteratively optimized to maintain the most efficient connections. As the number of iterations increases, an optimal path between $\mathbf{x}_{\text{start}}$ and $\mathbf{x}_{\text{goal}}$ emerges within the tree, yielding a solution that minimizes the specified cost function while satisfying dynamic constraints.

Kino-dynamic RRT*

Input: state space $\mathcal{X}$; $\mathbf{x}_{start} \in \mathcal{X}_{free}$; $\mathbf{x}_{goal} \in \mathcal{X}_{free}$

Output: path $\mathcal{T}$ from $\mathbf{x}_{start}$ to $\mathbf{x}_{goal}$

$\mathcal{T} \leftarrow \{\mathbf{x}_{start}\}$
for $i \in [1, \infty)$ **do**
  $\mathbf{x}_i \leftarrow$ randomSample $\{\mathcal{X}_{free}\}$
  `// Parent selection`
  $\mathbf{x}_{near} \leftarrow \underset{\mathbf{x} \in \mathcal{T}}{\text{argmin}} \; J(\mathbf{x}) + J^*(\mathbf{x}, \mathbf{x}_i)$
    s. t. $J^*(\mathbf{x}, \mathbf{x}_i) < r$
      collisionFree $(\pi^*(\mathbf{x}, \mathbf{x}_i))$
  $\mathcal{T}.parent(\mathbf{x}_i) \leftarrow \mathbf{x}_{near}$
  $J(\mathbf{x}_i) \leftarrow J(\mathbf{x}_{near}) + J^*(\mathbf{x}_{near}, \mathbf{x}_i)$
  `// Rewire tree`
  for all $\mathbf{x} \in \mathcal{T} \cup \{x_{goal}\}$ & $J^*(\mathbf{x}_i, \mathbf{x}) < r$ **do**
    if $J(\mathbf{x}_i) + J^*(\mathbf{x}_i, \mathbf{x}) < J(\mathbf{x})$ & collisionFree $(\pi^*(\mathbf{x}_i, \mathbf{x}))$ **then**
      $J(\mathbf{x}) \leftarrow J(\mathbf{x}_i) + J^*(\mathbf{x}_i, \mathbf{x})$
      $\mathcal{T}.parent(\mathbf{x}) \leftarrow \mathbf{x}_i$
  $\mathcal{T} \leftarrow \mathcal{T} \cup \{\mathbf{x}_i\}$

Algorithm 1: Pseudocode of the Kinodynamic RRT* algorithm. The tree $\mathcal{T}$ comprises states, each associated with a parent pointer, denoted as parent($\mathbf{x}$), and a cost value, cost($\mathbf{x}$), which represents the trajectory cost from the initial state. A trajectory segment is deemed feasible if it satisfies the condition collisionFree$(\mathbf{x}(t), \mathbf{u}(t), \tau) = \left\{\mathbf{x}(t) \in \mathcal{X}_{\text{free}},\ \mathbf{u}(t) \in \mathcal{U}_{\text{free}},\ \forall t \in [0, \tau]\right\}$, ensuring that both the state and control inputs remain within their respective collision-free spaces throughout the entire trajectory duration.

The implementation of the algorithm, as outlined in the corresponding pseudocode, introduces several modifications relative to the standard RRT* framework. A fundamental distinction lies in the problem formulation, which aims to determine a trajectory that precisely reaches a predefined goal state, rather

than merely arriving within a goal region, as is typically the case in conventional RRT* applications. To accommodate this requirement, the goal state is explicitly included in the set of candidate states for forward connections from newly sampled nodes, even if it has not yet been incorporated into the tree. This modification ensures that the algorithm actively considers direct connections to the goal state whenever feasible, thereby expediting the convergence process.
Moreover, traditional implementations of RRT* often employ a "steer" function that incrementally extends the tree toward a sampled state, typically resulting in partial trajectories in which only the endpoint of the extension is added as a new node. However, within the present formulation, generating partial trajectories while enforcing a predefined cost limit proves non-trivial due to the kinodynamic constraints of the system. Consequently, the proposed algorithm attempts to establish a full connection to each sampled state, directly incorporating the sampled state into the tree whenever a feasible trajectory exists. These modifications preserve the fundamental properties of RRT*, ensuring that the algorithm retains its asymptotic optimality while adapting to the specific requirements imposed by kinodynamic motion planning.

2.4 Environmental Constraints

In motion planning, environmental constraints are fundamental in ensuring both the feasibility and safety of trajectories. The environment in which a robotic system operates is often characterized by the presence of obstacles, which must be accurately represented and accounted for during the planning process. A structured representation of these obstacles enables efficient decision-making and ensures that computed trajectories remain free of collisions.
One of the most effective methods for modeling environmental constraints is through the use of a cost map, which assigns numerical values to different regions based on their traversability. This representation facilitates the implementation of motion planning algorithms that rely on heuristic evaluations or optimization techniques to determine feasible paths. A well-structured cost map not only identifies obstacles but also integrates safety margins and accounts for environmental uncertainties, ensuring that planned trajectories satisfy both geometric and dynamic constraints.
Since the cost associated with a given trajectory is already managed by the

previously discussed algorithm, the primary requirement is an efficient method for identifying no-fly zones. To achieve this, the simplest and most effective approach is to use a binary 0-1 cost map, which explicitly differentiates between navigable and non-navigable regions.

2.4.1 Costmap Definition

A 0-1 cost map is a binary representation of an environment used in motion planning to distinguish between traversable and non-traversable regions. In this framework, the workspace is discretized into a grid or a set of sampled points, where each point is assigned a cost of either 0 or 1. A value of 0 indicates that the point is free and can be safely traversed, whereas a value of 1 signifies an occupied region corresponding to an obstacle. This form of representation is particularly useful in path planning algorithms, as it enables efficient collision checking while maintaining a structured approach to environmental modeling.
The obstacles in the environment are defined using a continuous function that provides a smooth and differentiable representation of the terrain [11]. The function governing the obstacle height distribution is given by

$$h(x,y) = h_0 + \sum_{i=1}^{n} h_i \exp\left[-\left(\frac{x-x_{0,i}}{x_{s,i}}\right)^2 - \left(\frac{y-y_{0,i}}{y_{s,i}}\right)^2\right]. \tag{26}$$

In this formulation, h_0 represents the baseline height of the environment, ensuring that obstacles are modeled relative to a reference surface. The term h_i corresponds to the height of each individual obstacle, while the coordinates $(x_{0,i}, y_{0,i})$ define the center positions of these obstacles in the two-dimensional plane. The parameters $x_{s,i}$ and $y_{s,i}$ determine the slope of each obstacle along the respective axes, controlling how rapidly the obstacle height decays from its peak value.
By employing a sum of Gaussian functions, the obstacle representation achieves smooth transitions rather than abrupt discontinuities. This approach is advantageous in motion planning applications, as it facilitates numerical computations and allows for more realistic obstacle avoidance strategies. Unlike grid-based representations that rely on fixed obstacle shapes, this method provides a continuous cost function, making it particularly well suited for algorithms that require gradient-based optimizations or sampling-based motion planning.
The cost map is constructed by discretizing the workspace using a user-defined

sampling resolution along all three spatial dimensions, which include the horizontal coordinates x and y, as well as the vertical dimension corresponding to the terrain height $h(x, y)$. The resolution is determined by the step sizes Δx, Δy, and Δh, which define the spacing between consecutive sampled points in the respective directions. Smaller step sizes result in a finer resolution, increasing the precision of obstacle representation at the cost of higher computational complexity. Conversely, larger step sizes lead to a coarser map, reducing computational requirements but potentially compromising the accuracy of obstacle modeling.

Each sampled point in the cost map is classified based on its height relative to the obstacle function. If the height at a given point lies above or within a predefined threshold of the function $h(x, y)$, the point is marked as occupied by assigning it a cost of 1. Otherwise, it is classified as free space and assigned a cost of 0. This classification ensures that the motion planning algorithm can efficiently distinguish between feasible and infeasible regions, allowing for rapid evaluation of potential trajectories while maintaining adaptability based on the specific requirements of the application.

2.4.2 Obstacles

In order to ensure reliable obstacle avoidance, an additional safety margin is introduced into the cost map, effectively expanding the occupied regions around obstacles. This modification accounts for various factors, including the physical dimensions of the moving system, potential localization errors, and dynamic constraints that may affect the trajectory execution. The extended obstacle representation incorporates a safety radius, which creates a buffer zone around each obstacle, ensuring that planned trajectories maintain a minimum clearance from potential collision points.

The updated obstacle function, which includes the safety margin, is defined as

$$h_s(x, y) = h(x, y) + r_s. \tag{27}$$

Here, the parameter r_s represents the safety radius, which is chosen based on the specific characteristics of the robotic system and the level of uncertainty present in the environment. By increasing the height of the obstacle function, the cost map effectively inflates the obstacles, restricting the available free space and enforcing a stricter constraint on the trajectory planning process.

The motion planning algorithm must ensure that all feasible trajectories re-

main below the modified height function $h_s(x, y)$ to guarantee collision-free navigation. Any trajectory that intersects or exceeds this threshold is deemed infeasible, as it would result in a violation of the predefined clearance requirements. This approach provides a systematic way to enforce obstacle avoidance while maintaining flexibility in adjusting the safety margin based on real-world constraints.

The effectiveness of this methodology is illustrated in Figure 16, which visually represents the original obstacle height function $h(x, y)$, the expanded safety region defined by $h_s(x, y)$, and several potential trajectories. The figure highlights the distinction between valid and invalid paths, demonstrating the impact of the safety radius on motion planning decisions.

Trajectories that remain entirely outside the expanded safety region $h_s(x, y)$ are classified as acceptable, ensuring that the moving system can navigate the environment without the risk of collision. On the other hand, trajectories that penetrate the safety zone are rejected, as they would result in insufficient clearance from obstacles. This distinction is critical in practical applications, where even minor deviations from the planned path could lead to unsafe conditions.

By incorporating the safety radius into the cost map, the motion planning framework is able to account for uncertainties and system constraints, leading to more robust and reliable trajectory generation. The additional computational cost associated with expanding the obstacle regions is justified by the significant improvements in safety and feasibility, making this approach essential for real-world applications in autonomous navigation and robotic motion planning.

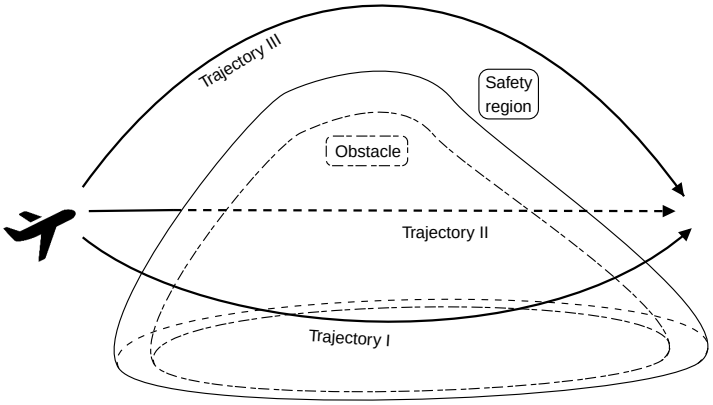

Figure 16: Obstacle representation with a safety radius, showing feasible and non-feasible trajectories [11].

2.5 Implementation Considerations

The practical implementation of motion planning algorithms requires careful consideration of various computational techniques to ensure efficiency, accuracy, and robustness. The effectiveness of an algorithm not only depends on its theoretical formulation but also on how its computational components are structured and optimized. Several mathematical and algorithmic strategies can be employed to enhance the numerical stability, computational complexity, and overall performance of path planning solutions.
This section presents key implementation considerations that are essential for optimizing the computational efficiency of the planning process.
Base remark: our system in Section 1.4 is clearly reachable, indeed

$$\mathcal{R} = \begin{bmatrix} \mathbf{G} & \mathbf{FG} & \dots \end{bmatrix} = \begin{bmatrix} \mathbf{0}_{3\times3} & \mathbf{I}_{3\times3} & \dots \\ \mathbf{I}_{3\times3} & \mathbf{0}_{3\times3} & \dots \end{bmatrix}$$

it's clearly reachable, i.e. rank $\mathcal{R} = 6$.
First, the use of companion matrices for polynomial root-finding is examined, providing a systematic approach to solving polynomial equations that arise in path planning computations. Next, the nilpotency property of the state transition matrix is explored, enabling the derivation of closed-form expressions for state and input evolution, which can significantly improve computational efficiency. Finally, the application of graph search algorithms, specifically breadth-first search (BFS) and depth-first search (DFS), is discussed in the context of motion planning, highlighting their roles in efficient exploration and path refinement. These considerations collectively contribute to the development of an implementation framework that ensures both computational feasibility and optimal performance.

2.5.1 Companion Matrix for Polynomial Root-Finding

In the fields of numerical linear algebra and computational mathematics, the task of determining the roots of a polynomial is a central problem with numerous applications across various scientific and engineering disciplines. Finding the roots of polynomials is essential for solving equations, analyzing system behavior, and performing optimizations. While there exist direct root-finding methods, a more structured and systematic approach involves the use of companion matrices. This method transforms the root-finding problem into an eigenvalue problem, which is advantageous because it can be efficiently solved

using well-established numerical methods for computing matrix eigenvalues.

Consider a polynomial of degree n in the variable x, expressed in its general form as:

$$p_1(x) = c_n x^n + c_{n-1} x^{n-1} + \cdots + c_1 x + c_0,$$

where $c_n, c_{n-1}, \ldots, c_1, c_0$ are the real or complex coefficients of the polynomial, and $c_n \neq 0$. This polynomial can be normalized by dividing by its leading coefficient, resulting in a polynomial of the form

$$p(x) = \frac{p_1(x)}{c_n} = x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0.$$

which is termed a monic polynomial. According to the *Fundamental Theorem of Algebra*, every polynomial of degree n has exactly n roots in the complex plane, counting multiplicities. Moreover, the solutions of $p(x)$ and $p_1(x)$ are the same. Specifically, if x_0 is a root of $p_1(x)$, meaning that $p_1(x_0) = 0$, then

$$p(x_0) = \frac{p_1(x_0)}{c_n} = 0$$

demonstrating that the process of normalization does not alter the roots of the polynomial. Thus, monic polynomials can be considered without loss of generality. To facilitate root search, the companion matrix provides a structured and effective approach. A companion matrix encodes the coefficients of a polynomial into a square matrix. For a monic polynomial $p(x)$ of degree n, the associated companion matrix C_x is an $n \times n$ matrix defined as follows:

$$C_x = \begin{bmatrix} 0 & 1 & 0 & 0 & \ldots & 0 \\ 0 & 0 & 1 & 0 & \ldots & 0 \\ 0 & 0 & 0 & 1 & \ldots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \ldots & 1 \\ -a_0 & -a_1 & -a_2 & -a_3 & \ldots & -a_{n-1} \end{bmatrix}.$$

This matrix exhibits a distinctive structure where the first $(n-1)$ rows consist of ones on the first superdiagonal and zeros elsewhere, while the last row consists of the negated coefficients $a_0, a_1, \ldots, a_{n-1}$ from the polynomial. The companion matrix thus serves as a transformation of the polynomial equation into matrix.

The eigenvalue problem is formally defined by the characteristic polynomial

of the matrix C_x, which is obtained as the determinant of the matrix $\lambda I - C_x$, where I represents the $n \times n$ identity matrix. The roots of this characteristic polynomial correspond to the eigenvalues of the matrix C_x.

Consider the formulation of the determinant:

$$\det(\lambda I - C_x) = \begin{vmatrix} \lambda & -1 & 0 & 0 & \dots & 0 \\ 0 & \lambda & -1 & 0 & \dots & 0 \\ 0 & 0 & \lambda & -1 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \dots & -1 \\ a_0 & a_1 & a_2 & a_3 & \dots & \lambda + a_{n-1} \end{vmatrix}$$

By applying the Laplace expansion along the first row, the new expression of the determinant becomes:

$$\det(\lambda I - C_x) = \sum_{j=1}^{n} (-1)^{1+j} a_{1j} \det(M_{1j}), \tag{28}$$

where a_{1j} are the elements of the first row of the matrix $\lambda I - C_x$, and M_{1j} represents the submatrix obtained by removing the first row and j-th column from the matrix. This expansion significantly simplifies the calculation of the determinant, since for the matrix $\lambda I - C_x$, the first row is:

$$\begin{bmatrix} \lambda & -1 & 0 & 0 & \dots & 0 \end{bmatrix}$$

hence, substituting in (28),the determinant becomes:

$$\det(\lambda I - C_x) = \lambda \det(M_{11}) - (-1) \det(M_{12}) + 0 \det(M_{13}) - 0 \det(M_{14}) + \dots$$

Here, $M_{11}, M_{12}, \dots$ are the submatrices obtained by removing the first row and the corresponding columns, and the terms involving zero determinants are omitted. The calculation of each submatrix's determinant follows a similar recursive procedure.

Consider the submatrix M_{11}, obtained by removing the first row and first

column of $\lambda I - C_x$. This submatrix has the following form:

$$M_{11} = \begin{bmatrix} \lambda & -1 & 0 & \dots & 0 \\ 0 & \lambda & -1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & -1 \\ a_1 & a_2 & a_3 & \dots & \lambda + a_{n-1} \end{bmatrix}$$

This submatrix has the same structure as the original matrix $\lambda I - C_x$, but with reduced size $(n-1) \times (n-1)$. Thus, the determinant of M_{11} follows the same pattern and will lead to the same type of polynomial, albeit with smaller degree. The recursive nature of this process continues as one progresses through all submatrices.

Similarly, for the submatrix M_{12}, obtained by removing the first row and the second column of $\lambda I - C_x$, it follows that:

$$M_{12} = \begin{bmatrix} 0 & -1 & 0 & \dots & 0 \\ 0 & \lambda & -1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & -1 \\ a_0 & a_2 & a_3 & \dots & \lambda + a_{n-1} \end{bmatrix}.$$

This submatrix also follows a similar structure, and its determinant will contribute to the overall characteristic polynomial of C_x.

The recursive process continues, with each submatrix having a similar structure, ultimately leading to a base case when a 1×1 matrix is reached. The resulting determinants at each step combine to produce the characteristic polynomial of the matrix C_x.

Finally, the determinant of the original matrix $\lambda I - C_x$ simplifies to the characteristic polynomial:

$$\det(\lambda I - C_x) = \lambda^n + a_{n-1}\lambda^{n-1} + \cdots + a_1\lambda + a_0$$

This characteristic polynomial is a degree n polynomial in λ, whose roots are the eigenvalues of the companion matrix C_x. Thus, the roots of the polynomial $p(x)$, given by the original problem, are precisely the eigenvalues of the companion matrix C_x.

The use of the companion matrix for polynomial root-finding provides a sys-

tematic and reliable approach to determining the roots of a polynomial by analyzing the eigenvalues of the corresponding matrix. This representation effectively transforms the root-finding problem into the well-established eigenvalue computation problem, which is supported by a wealth of research and optimization in numerical linear algebra. Eigenvalue solvers, such as the Eigen library in C++, are widely available and highly efficient, offering optimized performance in terms of both speed and stability. Unlike traditional polynomial root-finding methods, which can be sensitive to perturbations in the polynomial coefficients, the companion matrix method offers greater robustness and stability, as it is less prone to instability caused by small variations in the input data.

2.5.2 Nilpotency of the State Transition Matrix

In the feedback linearized model derived in Section 1.4, the state matrix $\mathbf{F}$ is nilpotent, meaning that $\exists\, k \in \mathbb{Z}_+$ such that $\mathbf{F}^k = \mathbf{0}$. Consequently, the matrix exponential reduces to

$$e^{\mathbf{F}t} = \sum_{i=0}^{\infty} \frac{t^i}{i!}\mathbf{F}^i = \mathbf{I}_{6\times 6} + \mathbf{F}t$$

and setting $\mathbf{R} = \mathbf{I}_{6\times 6}$, a closed-form expression for the weighted reachability Gramian in (15) is obtained:

$$\mathcal{W}_t = \begin{bmatrix} \frac{t^3}{3} & 0 & 0 & \frac{t^2}{2} & 0 & 0 \\ 0 & \frac{t^3}{3} & 0 & 0 & \frac{t^2}{2} & 0 \\ 0 & 0 & \frac{t^3}{3} & 0 & 0 & \frac{t^2}{2} \\ \frac{t^2}{2} & 0 & 0 & t & 0 & 0 \\ 0 & \frac{t^2}{2} & 0 & 0 & t & 0 \\ 0 & 0 & \frac{t^2}{2} & 0 & 0 & t \end{bmatrix}.$$

Substituting the latter into the autonomous evolution of the state equation (13), the state trajectory takes the form:

$$\bar{\mathbf{x}}(t) = \begin{bmatrix} x_{i,1} + tx_{i,4} \\ x_{i,2} + tx_{i,5} \\ x_{i,3} + tx_{i,6} \\ x_{i,4} \\ x_{i,5} \\ x_{i,6} \end{bmatrix}$$

and consecutively the terminal condition of the costate equation (19) is:

$$\mathbf{d}(\tau) = \begin{bmatrix} -\frac{6(2x_{i,1}-2x_{j,1}+\tau x_{i,4}+\tau x_{j,4})}{\tau^3} \\ -\frac{6(2x_{i,2}-2x_{j,2}+\tau x_{i,5}+\tau x_{j,5})}{\tau^3} \\ -\frac{6(2x_{i,3}-2x_{j,3}+\tau x_{i,6}+\tau x_{j,6})}{\tau^3} \\ \frac{2(3x_{i,1}-3x_{j,1}+\tau x_{i,4}+2\tau x_{j,4})}{\tau^2} \\ \frac{2(3x_{i,2}-3x_{j,2}+\tau x_{i,5}+2\tau x_{j,5})}{\tau^2} \\ \frac{2(3x_{i,3}-3x_{j,3}+\tau x_{i,6}+2\tau x_{j,6})}{\tau^2} \end{bmatrix}.$$

This formulation results in a closed-form expression for $\dot{J}(\tau)$ which simplifies to a polynomial in one variable. The roots of this polynomial can be efficiently determined by computing the eigenvalues of the associated companion matrix. The optimal time τ^* is then determined and substituted into (19) to compute $\mathbf{d}(\tau^*)$. With this information, the equation governing the state evolution of the optimal trajectory (25) is established as:

$$\mathbf{x}(t) = \begin{bmatrix} x_{j,1} + x_{j,4}(t-\tau^*) - \frac{d_1(\tau^*)(t-\tau^*)^3}{6} + \frac{d_4(\tau^*)(t-\tau^*)^2}{2} \\ x_{j,2} + x_{j,5}(t-\tau^*) - \frac{d_2(\tau^*)(t-\tau^*)^3}{6} + \frac{d_5(\tau^*)(t-\tau^*)^2}{2} \\ x_{j,3} + x_{j,6}(t-\tau^*) - \frac{d_3(\tau^*)(t-\tau^*)^3}{6} + \frac{d_6(\tau^*)(t-\tau^*)^2}{2} \\ x_{j,4} + d_4(\tau^*)(t-\tau^*) - \frac{d_1(\tau^*)(t-\tau^*)^2}{2} \\ x_{j,5} + d_5(\tau^*)(t-\tau^*) - \frac{d_2(\tau^*)(t-\tau^*)^2}{2} \\ x_{j,6} + d_6(\tau^*)(t-\tau^*) - \frac{d_3(\tau^*)(t-\tau^*)^2}{2} \end{bmatrix}.$$

The corresponding control input (25) is given by:

$$\mathbf{u}(t) = \begin{bmatrix} d_4(\tau^*) - d_1(\tau^*)(t-\tau^*) \\ d_5(\tau^*) - d_2(\tau^*)(t-\tau^*) \\ d_6(\tau^*) - d_3(\tau^*)(t-\tau^*) \end{bmatrix}.$$

Finally, the closed-form expression for the optimal cost (9) is obtained as:

$$J^*(\mathbf{x}_i, \mathbf{x}_j) = \tau^* + \frac{\tau^*}{3}\left(\sum_{i=1}^{3}(\tau^*)^2 d_i^2(\tau^*) + 3\sum_{i=1}^{3}\tau^* d_i(\tau^*)d_{i+3}(\tau^*) + 3\sum_{i=1}^{3} d_{i+3}^2(\tau^*)\right).$$

By exploiting the nilpotency property of the state transition matrix, closed-form expressions are derived for the evaluation of the cost function, along with the state and input equations. This analytical formulation enables a more structured and systematic approach to the numerical computation of the optimal trajectory, increasing computational efficiency and reducing the

complexity associated with iterative numerical solvers.

2.5.3 Breadth-First Search and Depth-First Search

Breadth-First Search (BFS) and Depth-First Search (DFS) are fundamental algorithms for graph traversal, widely used to explore nodes and edges within graph or tree structures. Both algorithms are designed to visit all reachable nodes from a given starting point; however, they employ different traversal strategies that result in distinct node visitation orders. These differences in traversal mechanisms are illustrated in Figure 17, which provides a visual comparison of the BFS and DFS operations on a sample tree structure, clearly highlighting their contrasting approaches to node exploration.

Breadth-First Search (BFS) is an algorithm that systematically explores all nodes at a given depth level before proceeding to the nodes at the next depth level. This level-by-level exploration is made possible by using a queue data structure, which ensures that nodes are visited in the order in which they are discovered. The BFS algorithm initiates by selecting a starting node, marking it as visited, and enqueuing it into a queue. The algorithm then processes nodes by dequeuing the front node from the queue, marking it as visited, and subsequently enqueuing all of its unvisited adjacent nodes. This process repeats iteratively, with nodes being visited level by level, until the queue is empty, signaling that all reachable nodes have been processed.

The time complexity of BFS is $O(V + E)$, where V represents the number of vertices (or nodes) in the graph, and E denotes the number of edges. This reflects the need to visit every vertex and edge in the graph. The space complexity is $O(V)$, as, in the worst case, the queue may store all the vertices in the graph simultaneously, particularly in cases of wide graphs with many nodes at each level.

Depth-First Search (DFS), in contrast, explores the graph by traversing as deeply as possible along each branch before backtracking to explore alternate paths. This deep traversal is facilitated by a stack data structure, which can be implemented either explicitly, by maintaining a stack data structure, or implicitly, through the recursive nature of the algorithm. The DFS algorithm starts by selecting an initial node, marking it as visited, and pushing it onto the stack (or initiating a recursive call). The algorithm then pops the top node from the stack, processes it, and pushes all unvisited adjacent nodes onto the stack (or recursively visits them). This process continues until the stack is empty or recursion terminates, signifying that all reachable nodes

have been visited.
DFS is often more memory-efficient than BFS, especially in cases where the graph or tree is deep, because it only needs to store the nodes along the current path from the root to the current node in memory. The time complexity of DFS is also $O(V + E)$, similar to that of BFS, as every vertex and edge must be examined. However, the space complexity of DFS is generally $O(V)$ in the worst case, though in practice it tends to be lower due to the algorithm's depth-first exploration. In cases of shallow trees or graphs, DFS may require storing only a small number of nodes at any given point in the traversal.

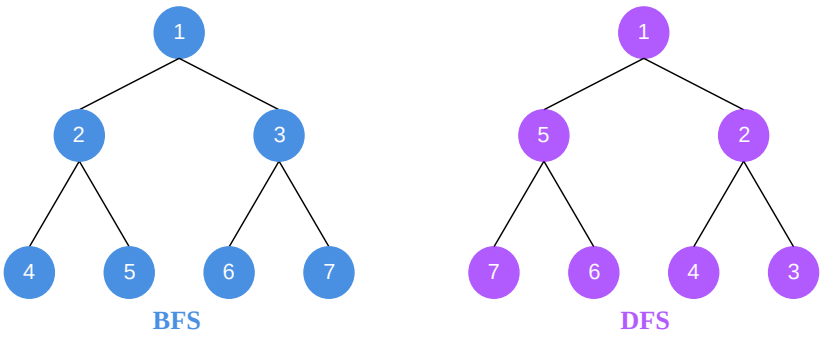

Figure 17: A visualization of Breadth-First Search (left) and Depth-First Search (right), illustrating the order in which nodes are visited by each algorithm [15].

Referring to the algorithm presented in Section 2.3.5, the primary design choices involve using DFS for parent selection and BFS for tree rewiring. These choices are motivated by computational efficiency and the structured evaluation of node connections.
In the parent selection process, the objective is to identify the nearest valid parent node $\mathbf{x}_{\text{near}}$ that minimizes the cost function:

$$J(\mathbf{x}) + J^*(\mathbf{x}, \mathbf{x}_i)$$

for every node $\mathbf{x}$ in the tree, while ensuring that the radius constraint

$$J^*(\mathbf{x}, \mathbf{x}_i) < r$$

and the feasibility condition

$$\text{collisionFree}\left(\pi^*(\mathbf{x}, \mathbf{x}_i)\right)$$

are satisfied. Since the cost is generally proportional to the distance, solv-

ing this optimization problem efficiently requires maintaining an auxiliary variable to store the lowest cost encountered during the search. Given that cost evaluation is computationally cheaper than feasibility verification, DFS is employed to rapidly explore deeper paths in the tree, allowing for early identification of lower-cost connections. By quickly refining the minimum cost stored in the auxiliary variable, subsequent branches can be pruned more aggressively, reducing the number of feasibility checks required. In the worst-case scenario, where no early pruning occurs, DFS spans the entire tree with the same computational complexity as BFS.
For the rewiring phase, BFS is selected as it efficiently processes nodes in increasing order of distance, ensuring a systematic update of connections. Since the rewiring procedure already incorporates two fast preliminary checks—the neighborhood condition

$$J^*(\mathbf{x}_i, \mathbf{x}) < r$$

and the rewiring condition

$$J(\mathbf{x}_i) + J^*(\mathbf{x}_i, \mathbf{x}) < J(\mathbf{x})$$

before evaluating feasibility, BFS provides an effective balance between computational cost and structured node exploration.

3 Results and Analysis

This chapter presents the results of both numerical simulations and real-world experiments, accompanied by a discussion of their implications. The objective is to evaluate the performance of the proposed approach in both simulated and physical environments, ensuring consistency between theoretical predictions and practical implementation.

The first section focuses on MATLAB-based simulations, which serve as an initial step to gain deeper insights into the underlying theoretical framework and to verify the fundamental logic of the algorithm introduced in Section 2.3.5. These simulations provide a controlled environment in which various scenarios can be tested and analyzed, facilitating a systematic approach to refining the algorithm before moving to more complex settings.

Subsequently, an overview of the ARPL architecture [16] is provided. This discussion is essential for understanding how more realistic simulations are conducted and how the experimental setup for real-world testing is established. The integration of this architecture allows for a closer approximation to practical conditions, bridging the gap between theoretical modeling and actual deployment.

For parameter selection, constraints derived from a real fixed-wing aircraft were considered. Specifically, the roll and pitch angles were limited to

$$-40^\circ < \phi < 40^\circ$$
$$-10^\circ < \gamma < 10^\circ$$

respectively. To ensure that these bounds were respected, a dedicated script was developed to determine the corresponding constraints on the control inputs of the feedback linearized model, formulated in Section 1.4. The computed input bounds are given by:

$$\begin{bmatrix} -8.29 \\ -8.29 \\ -7.72 \end{bmatrix} \leq \mathbf{u} \leq \begin{bmatrix} 8.29 \\ 8.29 \\ 1.46 \end{bmatrix}$$

This procedure ensures that the control inputs remain within feasible limits, thereby maintaining adherence to the physical constraints of the system.

3.1 Matlab Simulations

To evaluate the performance and feasibility of the proposed motion planning approach, MATLAB simulations were conducted under two distinct scenarios: one involving a single obstacle and another incorporating three obstacles. The primary objective was to analyze the trajectory generation process and assess whether the computed paths adhered to the given constraints. The algorithm was executed for 1000 iterations without imposing a time limit. The radius cost was set to $r = 150$, with a safety radius of $r_s = 10$ meters, while the velocity was constrained within the range of $15 \leq V \leq 30\ [m/s]$.
For the scenario with one obstacle, the resulting trajectory is illustrated in Figure 18.

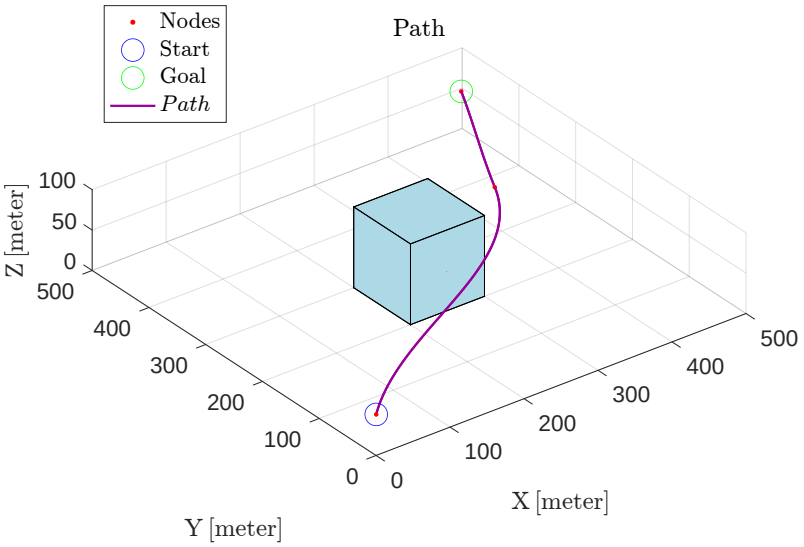

Figure 18: 3D view of the planned path for the fixed-wing aircraft with a single obstacle.

To further analyze the system's behavior, Figure 19 presents the position and velocity components along the x, y, and z axes with respect to time.
Additionally, the roll and pitch angles, which characterize the attitude of the system during motion, are depicted in Figure 20.
Lastly, Figure 21 displays the cost function evolution and input signals, providing insights into the optimization process and control inputs applied throughout the trajectory.
To assess the algorithm's performance in a more constrained environment, a second simulation was conducted with three obstacles.
The generated path is visualized in Figure 22, while the time evolution of position and velocity components is provided in Figure 23.

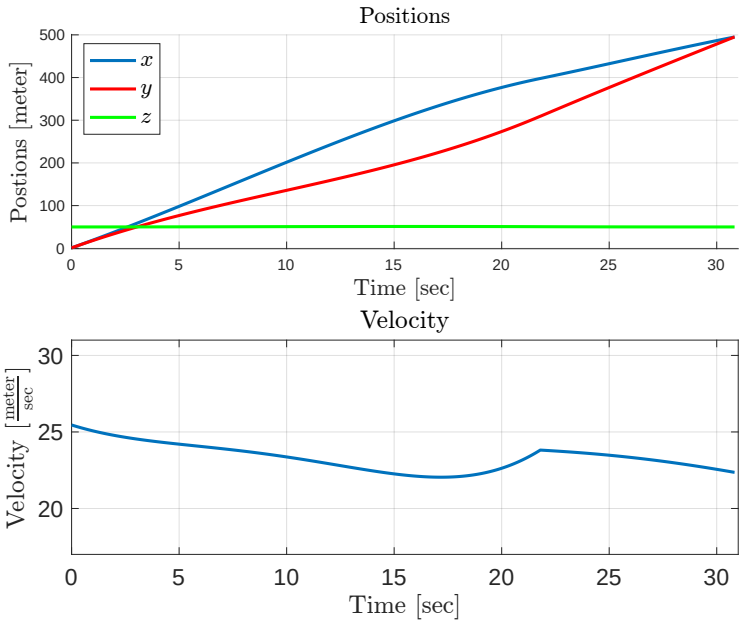

Figure 19: Position (in x, y, z) in the upper plot, and velocity in the lower plot for simulation with a single obstacle.

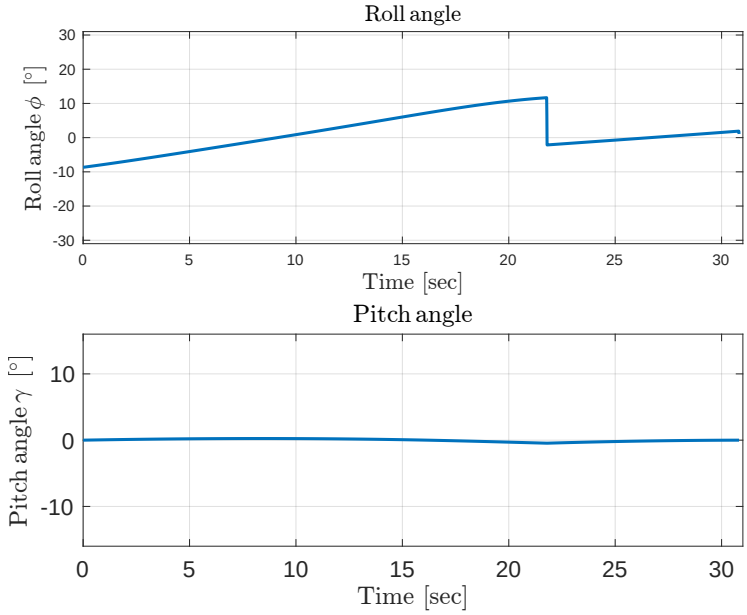

Figure 20: Roll angle ϕ in the upper plot and pitch angle γ in the lower plot for simulation with a single obstacle.

The roll and pitch angles are plotted in Figure 24, whereas Figure 25 illustrates the decreasing cost function and the corresponding input signals.
The results indicate that the computed trajectories respect the imposed constraints on velocity, roll, and pitch angles, as well as input limits, as evident from Figures 19, 20, and 21, with a more pronounced confirmation in Figures 23, 24, and 25. However, a noticeable issue arises regarding the smoothness

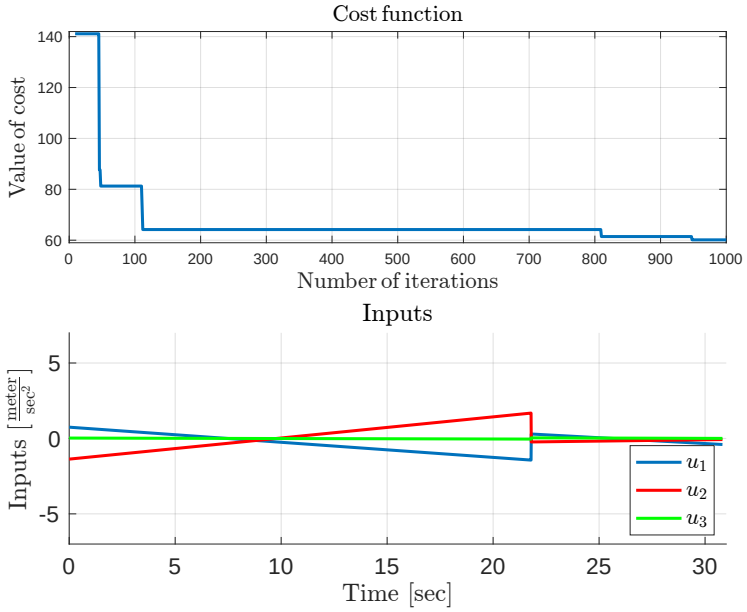

Figure 21: Cost function in the upper plot and input signals in the lower plot for the simulation with a single obstacle.

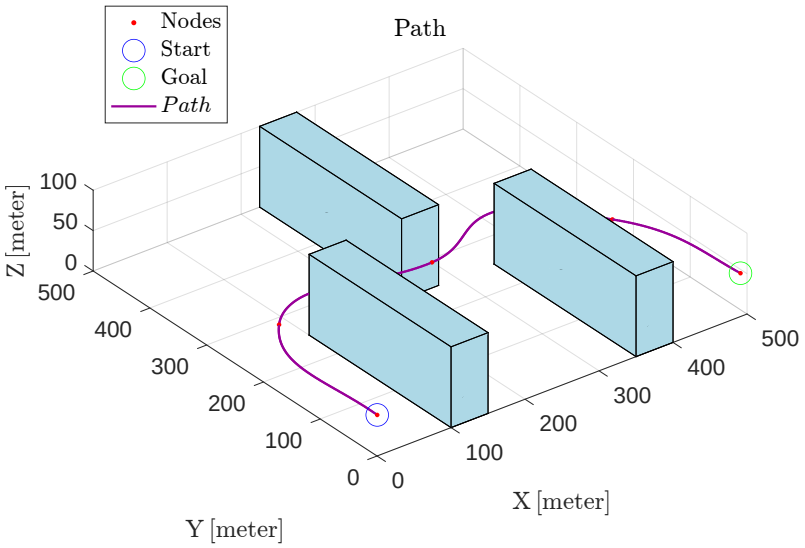

Figure 22: 3D view of the planned path for the fixed-wing aircraft with three obstacles.

of the generated trajectories. The presence of discontinuities, particularly at the interconnections between consecutive nodes, suggests that the algorithm does not inherently enforce smooth transitions. This can pose a significant challenge in real-world applications, as abrupt changes in velocity, roll, pitch, or control inputs may lead to mechanical strain or instability, especially in systems relying on physical actuators such as rudders or ailerons.

Moreover, the cost function plots in Figures 21 and 25 provide insights into

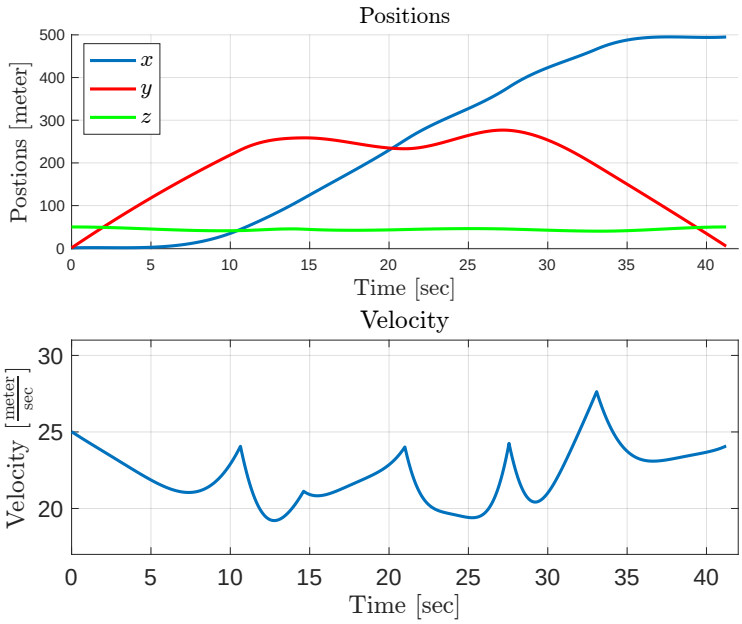

Figure 23: Position (in x, y, z) in the upper plot, and velocity in the lower plot for simulation with three obstacles.

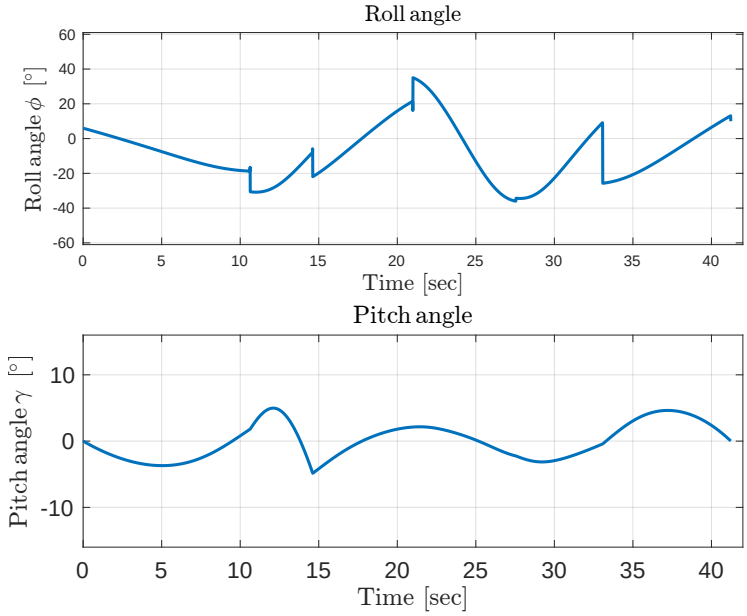

Figure 24: Roll angle ϕ in the upper plot and pitch angle γ in the lower plot for simulation with three obstacles.

the feasibility of the solutions. The absence of cost values at certain iterations suggests that a feasible path had not yet been found at those stages. However, with sufficient iterations, the cost function exhibits a decreasing trend, demonstrating that the algorithm successfully refines the trajectory over time. Notably, in the presence of multiple obstacles, the time required to determine an initial feasible path and subsequently reduce its cost is significantly higher.

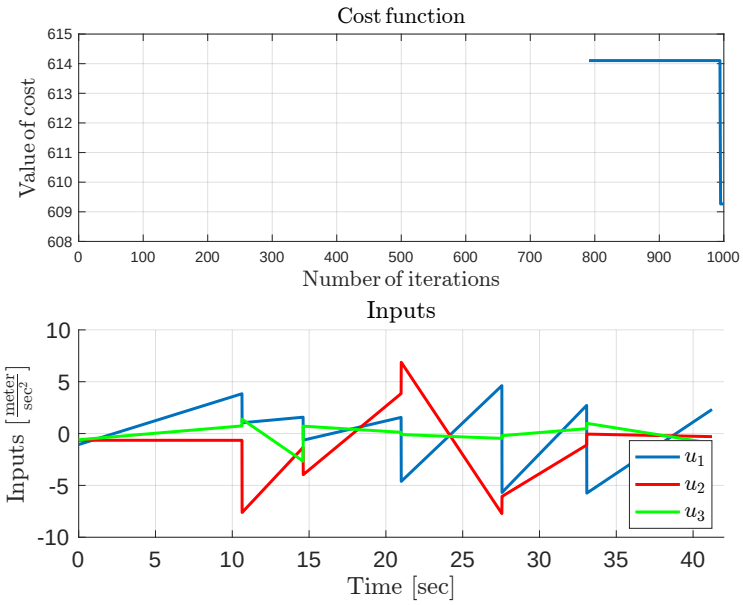

Figure 25: Cost function in the upper plot and input signals in the lower plot for the simulation with three obstacles.

This increased complexity arises from the constrained nature of the environment, where finding an optimal trajectory requires additional computational effort.

The primary objective of these simulations was to validate the feasibility of the proposed approach rather than achieving an optimized implementation. Several computational limitations were encountered, primarily due to the choice of numerical methods. Specifically, the MATLAB Symbolic Toolbox was employed instead of closed-form solutions, and MATLAB's standard root-finding functions were utilized rather than more efficient alternatives. These decisions resulted in increased computational time, limiting the efficiency of the implementation.

Nonetheless, despite these drawbacks, the results confirm that the fundamental approach is valid for the intended application. Future improvements can be made by incorporating the implementation considerations discussed in Section 2.5, which could significantly enhance computational efficiency and trajectory smoothness. By refining the numerical methods, enforcing continuity constraints, and improving obstacle avoidance strategies, the overall performance of the algorithm can be further optimized to meet the demands of real-world applications.

3.2 ARPL Architecture

The architectural framework utilized for the fixed-wing aircraft, as developed by the Agile Robotics and Perception Lab[1], is depicted in Figure 26. This architecture establishes a structured pipeline for autonomous flight, integrating multiple subsystems that collectively enable real-time perception, motion planning, and control.

The trajectory generation process employed in this architecture is based on Bernstein polynomials, a well-known mathematical representation that provides smooth and continuous trajectory generation. Bernstein polynomials offer advantageous properties, such as inherent smoothness and flexibility, making them suitable for trajectory optimization in dynamic environments.

For flight stabilization and trajectory tracking, the control module within the architecture implements a Proportional-Integral-Derivative (PID) controller. The PID control strategy is adopted due to its simplicity and effectiveness in maintaining system stability and achieving tracking performance. By continuously adjusting control inputs based on position and velocity errors, the PID controller ensures that the aircraft follows the planned trajectory with minimal deviation.

Furthermore, the architecture defines the position of the aircraft in $\mathbb{R}^3$ as the differential flat output of the system [17]. Differential flatness is a key theoretical property that simplifies trajectory generation and control by expressing the system's states and inputs in terms of a set of flat outputs and their derivatives. In the context of fixed-wing aircraft, defining the position as a flat output enables the direct computation of feasible control inputs required to achieve a desired trajectory.

The entire pipeline is implemented in C++, exploiting the implementation considerations discussed in Section 2.5. This choice ensures high computational efficiency and high performance, facilitating seamless integration across hardware and simulation environments.

PX4 Autopilot is an open-source flight control software stack designed for unmanned aerial vehicles (UAVs) and other robotic platforms. It provides a robust framework for controlling fixed-wing aircraft, supporting various hardware platforms and integrating with multiple ground control stations, sensors, and communication protocols. PX4 is widely used in research due to its modularity, adaptability, and extensive documentation. It operates using a real-time operating system (RTOS) on embedded hardware, executing essential

[1] `https://wp.nyu.edu/arpl/`

flight control loops at high frequency. The system features state estimation, navigation, and control algorithms that work in conjunction with external sensors, such as GPS, IMUs, and barometers, to maintain stability and autonomy. The software architecture is designed to support multiple vehicle types, allowing users to configure and customize it for specific applications.
QGroundControl (QGC) is the official ground control station (GCS) software for PX4-based systems. It provides a graphical user interface (GUI) for mission planning, parameter tuning, real-time telemetry monitoring, and firmware updates. QGC facilitates direct communication with the PX4 autopilot via MAVLink, enabling seamless integration between the operator and the UAV. Its multi-platform support allows it to run on Windows, macOS, Linux, and mobile devices, ensuring versatility in different operating environments.
To support both real-world experiments and simulation-based testing, the architecture is also connected to a Software-in-the-Loop (SITL) simulator in Gazebo. This configuration provides the flexibility to seamlessly switch between hardware experiments with the real aircraft and simulated testing within a controlled virtual environment. The SITL setup ensures rigorous validation of algorithms before real-world deployment, reducing potential risks and optimizing system performance.
The hardware used in this setup consists of several key components essential for autonomous fixed-wing UAV operations. These components include the airframe, autopilot, onboard computing system, and localization sensors.
The UAV platform selected for this implementation is the Hobby King Bixler 3 model, a lightweight and durable fixed-wing aircraft commonly used for research and development purposes. The Bixler 3 offers a stable aerodynamic design, making it suitable for autopilot integration and autonomous flight experiments. Its ample fuselage space allows for the installation of additional hardware components, including avionics, sensors, and payloads.
The primary flight controller utilized in this setup is the Pixhawk autopilot, specifically the Holybro® PX4[2]. The Pixhawk serves as the low-level attitude controller, responsible for executing flight stabilization, navigation, and real-time motor actuation. It integrates with PX4 firmware, providing access to a wide range of flight control features, including PID tuning, failsafe mechanisms, and sensor fusion algorithms.
The Pixhawk autopilot processes real-time sensor data from onboard IMUs,

[2] `https://px4.io/`

barometers, and GNSS receivers, ensuring precise attitude and position control. It communicates with external modules and onboard computing systems via MAVLink, enabling seamless data exchange and mission execution.

Accurate localization is crucial for autonomous UAV operations. This implementation utilizes the Drotek® F9P GNSS receiver module, which is integrated with PX4's Extended Kalman Filter (EKF2)-based state estimator. The F9P receiver provides high-precision GPS data with real-time kinematic (RTK) corrections, ensuring centimeter-level accuracy in position estimation. The state estimator fuses GNSS data with IMU and barometer readings to deliver robust state estimation, allowing the UAV to perform precise waypoint navigation and autonomous landings. The EKF2 module optimally combines sensor data to enhance the reliability of position and velocity estimates, even in environments with GNSS signal degradation. The integration of multiple sensors ensures redundancy, improving robustness against temporary sensor failures or signal loss.

To manage high-level decision-making and computationally intensive tasks, a dedicated onboard computing unit is employed to ensure efficient processing and real-time system responsiveness. The selected hardware platform is the NVIDIA® Jetson Xavier Orin board, which operates on Ubuntu 20.04 and leverages the Robot Operating System (ROS)[3] framework. This combination provides a robust and flexible software environment that supports modular development, facilitating seamless integration of various functional components.

The use of ROS enables structured intra-process communication, ensuring efficient data exchange between different software modules, including perception, state estimation, motion planning, and control. This architecture allows for real-time interaction between multiple onboard processes, ensuring that critical computations are executed with minimal latency. Furthermore, ROS provides a well-established ecosystem for robotic applications, offering a wide range of libraries and tools that enhance the overall system's adaptability and extensibility.

In this setup, the Jetson Xavier Orin board plays a crucial role in executing the online path planning algorithm, which determines feasible and optimal trajectories based on real-time environmental data and mission objectives. The platform is responsible for generating the trajectory iteratively while adapting to dynamic changes in the environment, ensuring smooth navigation

[3] `www.ros.org`

and obstacle avoidance. Additionally, it performs continuous computation of control signals, which are derived from the processing of odometry and sensor data.

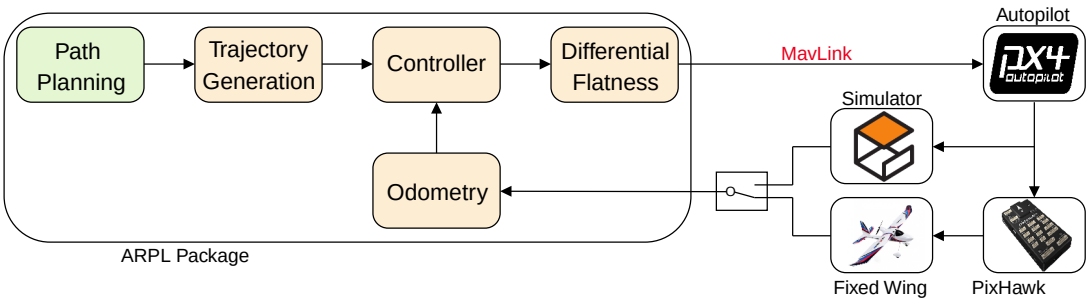

Figure 26: Block diagram of the ARPL pipeline for the fixed-wing aircraft, integrated with the PX4 autopilot.

The simulation environment utilized for the validation of the proposed approach is an outdoor flying arena located at Calvert Park in New York City, which also serves as the designated site for real-world flight testing. This environment provides a representative setting for evaluating the system's performance under realistic operational conditions, including external disturbances such as wind variations and sensor noise. The simulation framework is designed to replicate the real-world test conditions as accurately as possible, ensuring a consistent transition from virtual validation to physical deployment.

The initial phase of testing, conducted both in simulation and in real-world experiments, involves scenarios with static obstacles, identical to those used in preliminary MATLAB-based simulations. Subsequently, a more complex cost representation is employed, utilizing a 0-1 costmap, as elaborated in Section 2.4. This costmap is derived from the previously defined environment and incorporates simulated Gaussian distributions to model terrain elevation and obstacle density with the baseline of the costmap is set $h_0 = 0$. The spatial resolution of the costmap is configured by the step sizes $\Delta x = \Delta y = \Delta h = 1$, such that each grid cell represents a volume of 1 $[m^3]$ across the three spatial dimensions. This resolution provides a balance between computational efficiency and the fidelity required for accurate path planning.

As discussed in Section 1.5, the fixed-wing aircraft inherently lacks the ability to hover. To emulate a hovering-like behavior within the constraints of fixed-wing aerodynamics, the system employs a loitering maneuver, wherein the aircraft follows a circular trajectory. Consequently, the mission design consists

of two fundamental trajectory components: circular loitering and planned paths generated using the algorithm described in Section 2.3.5.

Specifically, the mission comprises two circular holding patterns, each with a radius of 45 $[m]$. The operational objective is to maintain the aircraft within a designated loitering circle until a feasible trajectory connecting the current loitering position to the next designated waypoint is computed. Once a valid path is determined, the aircraft exits the holding pattern and follows the planned trajectory until it reaches the second loitering position, where it re-enters circular flight until further navigation updates are available.

Throughout both simulations and real-world flight tests, the aircraft maintains a general operational altitude range of 35 to 45 meters above ground level. The atmospheric conditions during testing include wind speeds varying between 1 and 5 $[m/s]$, with the wind direction subject to random fluctuations over time. These environmental variations introduce realistic disturbances, allowing for a comprehensive assessment of the system's robustness under dynamic conditions.

To enhance computational efficiency while maintaining solution quality, the simulation framework incorporates specific termination criteria for the path planning algorithm. A maximum iteration limit of 500 is imposed, alongside a maximum runtime constraint of 300 seconds. Furthermore, once an initial feasible path is identified, the algorithm enters a refinement phase, during which a secondary termination condition is enforced. In this phase, a maximum of 50 additional iterations is allowed, with a predefined threshold of 0.1 applied to the cost function. If, within these iterations, the trajectory cost does not decrease by at least this threshold, the algorithm is terminated, and the most recently computed path is retained as the final solution. This approach ensures that computational resources are utilized efficiently while preventing unnecessary prolongation of the optimization process when no significant improvements can be achieved.

The roll and pitch constraints, as well as input limitations, are maintained consistently with those used in the MATLAB simulations. However, the velocity is constrained within the range of $10 \leq V \leq 25$ $[m/s]$ to ensure both stability and adherence to fixed wing structural limits.

To enhance trajectory optimization, an adaptive strategy is implemented for dynamically adjusting the radius cost parameter r, which plays a critical role in the path planning process. Initially, the optimal cost-to-go from the start node to the goal node, denoted as $J^*(\mathbf{x}_{\text{start}}, \mathbf{x}_{\text{goal}})$, is computed under the

assumption of an obstacle-free environment. This initial cost estimate is then used to define the starting radius cost as

$$r = 5 \cdot J^*(\mathbf{x}_{\text{start}}, \mathbf{x}_{\text{goal}}).$$

As the trajectory planning algorithm progresses, the radius cost is iteratively refined. Specifically, upon identifying the first feasible trajectory, the radius cost is updated to

$$r = J(\mathbf{x}_{\text{goal}}),$$

where $J(\mathbf{x}_{\text{goal}})$ represents the current cost associated with reaching the goal state. This process is repeated iteratively whenever a new trajectory with a lower cost is identified, ensuring continuous optimization of the planned path as better solutions emerge. By dynamically adjusting constraints throughout execution, this adaptive strategy enhances both the efficiency and effectiveness of the trajectory planning algorithm.

3.2.1 Simulations

The effectiveness of the proposed path planning algorithm was initially assessed within a simulated environment containing three obstacles. A three-dimensional representation of the scenario is provided in Figure 27, while a two-dimensional projection is depicted in Figure 28. Additionally, Figure 29 presents the interface of the QGroundControl software.

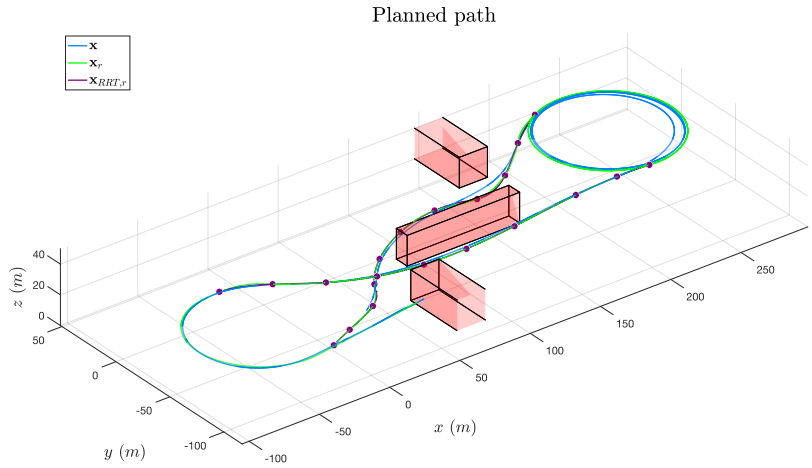

Figure 27: 3D view of the simulation with three obstacles. The fixed-wing position is shown in blue, the trajectory in green, and the planned path in purple.

The results indicate that the algorithm successfully converges to a collision-free trajectory while ensuring adherence to the kinematic and dynamic constraints of the fixed-wing aircraft. The computed path effectively avoids obstacles while maintaining smooth transitions between waypoints, thereby contributing to improved flight stability and control. These findings validate the algorithm's capability to generate feasible paths in constrained environments.

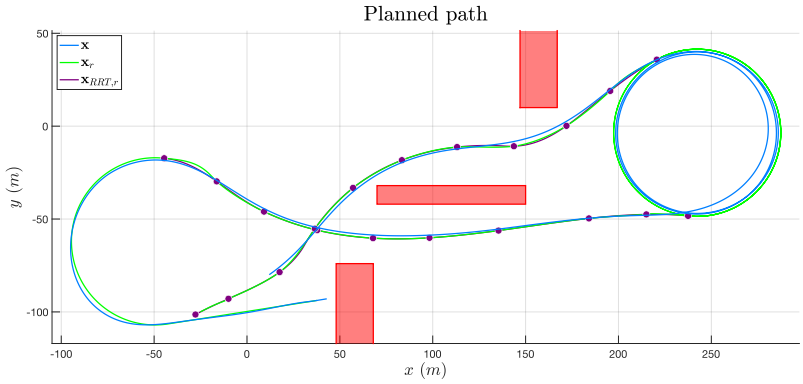

Figure 28: 2D view of the simulation with three obstacles. The fixed-wing position is shown in blue, the trajectory in green, and the planned path in purple.

Figure 29: qGroundControl interface for the simulation with three obstacles.

The computational time required for trajectory generation was systematically analyzed across multiple scenarios characterized by varying obstacle densities. This evaluation aims to assess the impact of environmental complexity on the feasibility of real-time and online motion planning.

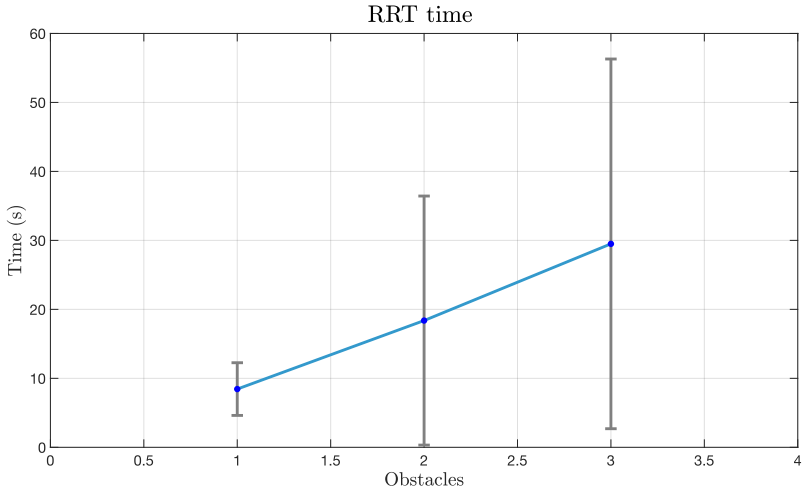

Figure 30: Computational time for simulations involving one, two, and three obstacles, represented in blue, with mean values of 8.88, 18.38, and 29.49 seconds, respectively. The corresponding confidence intervals, 3.82, 18.05, and 26.79 seconds, are shown in gray.

Figure 30 illustrates the computational time as a function of the number of obstacles present in the environment, alongside their corresponding confidence intervals. The data were collected from ten independent simulations for each obstacle count to ensure statistical reliability. As expected, an increase in the number of obstacles between the initial and goal positions leads to a general rise in computational time. However, a closer examination of the confidence intervals reveals that the spatial distribution of these obstacles also plays a significant role in determining the computational burden. Specifically, certain obstacle configurations may result in disproportionately higher computation times, whereas others may allow for more efficient trajectory generation despite a similar obstacle count.

Given these findings, achieving real-time planning remains a significant challenge, except in relatively simple environments with minimal obstructions. Nevertheless, online planning for subsequent trajectory segments remains feasible, as the required computational time remains within acceptable limits for onboard execution in real-world applications. This suggests that trajectory updates can be computed dynamically during the execution of a previously planned trajectory, enabling practical implementation in time-constrained scenarios.

The proposed path planning algorithm was further evaluated in a simulated environment utilizing the 0-1 costmap representation, as detailed in Section 2.4. This costmap formulation enables a discrete representation of the

environment, facilitating efficient trajectory optimization while preserving obstacle constraints. The corresponding three-dimensional visualization of the simulation scenario is presented in Figure 31, while the parameters defining the Gaussian distributions used to generate the costmap are summarized in Table 1.

$x_{0,i}$	$y_{0,i}$	h_i	$x_{s,i}$	$y_{s,i}$
-12	-91	40	5	5
15	-62	40	15	15
33	-108	40	20	20

Table 1: Values of the Gaussians in the 0-1 costmap for the simulation.

The results demonstrate that the algorithm successfully identifies a feasible and collision-free trajectory within the costmap-defined environment. The computed path adheres to the kinematic and dynamic constraints of the fixed-wing aircraft, avoiding regions with cost one while ensuring smooth transitions between waypoints. These findings confirm the algorithm's capability to navigate structured environments effectively, leveraging the costmap representation to refine path selection.

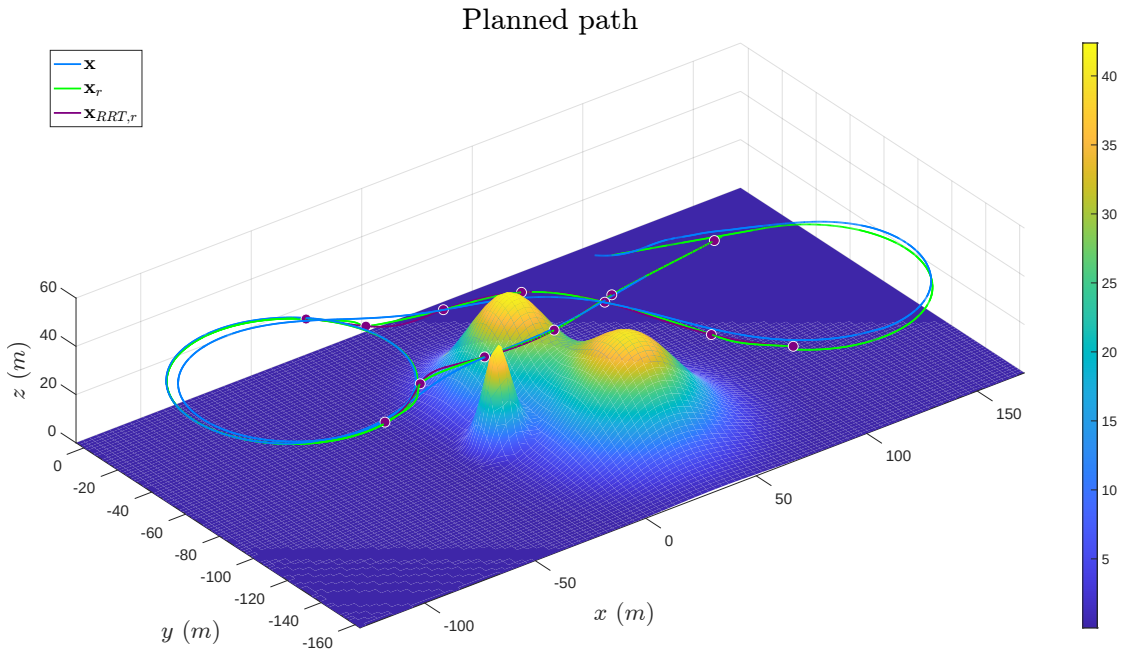

Figure 31: 3D view of the simulation with three gaussians. The fixed-wing position is shown in blue, the trajectory in green, and the planned path in purple.

3.2.2 Real Worlds Test

The proposed path planning algorithm was tested in real-world conditions, starting with an initial simple test scenario featuring a single obstacle. The corresponding two-dimensional visualization of this test setup is presented in Figure 32, with the QGC interface shown in Figure 33.

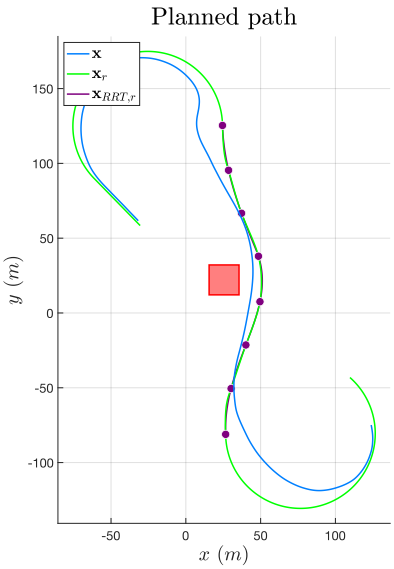

Figure 32: 2D view of the real test with a single obstacle. The fixed-wing position is shown in blue, the trajectory in green, and the planned path in purple.

This test was conducted primarily to assess the general functionality of the algorithm in a straightforward environment. The algorithm successfully generated a collision-free trajectory, demonstrating its capability to plan paths while adhering to the fixed-wing aircraft's kinematic constraints. Given the simplicity of the scenario, the performance was as expected, with the planned trajectory smoothly avoiding the obstacle.
Following this preliminary evaluation, a more complex test was conducted using the 0-1 costmap representation, as discussed in Section 2.4, and incorporating three obstacles. The three-dimensional visualization of this test scenario is provided in Figure 34, while the parameters for the Gaussian distributions used in the costmap are summarized in Table 2. In this test, the tracking performance was impacted by real-world conditions, such as sensor noise, flight dynamics, and environmental factors, resulting in a less precise path tracking compared to the ideal simulation.
Nevertheless, it is worth noting that both in the simulations and in the real-

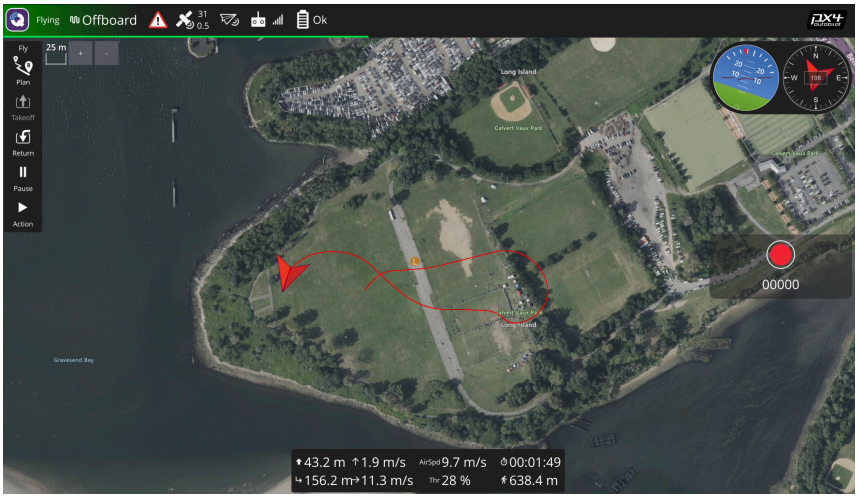

Figure 33: qGroundControl interface for the real word test with a single obstacle.

$x_{0,i}$	$y_{0,i}$	h_i	$x_{s,i}$	$y_{s,i}$
5	25	50	5	5
15	-20	50	15	15
43	15	50	15	15

Table 2: Values of the Gaussians in the 0-1 costmap for the real test.

world tests, the kinodynamic RRT* algorithm consistently outperformed the traditional circular loiter trajectory in terms of path tracking accuracy. This suggests that the proposed algorithm provides more optimal and feasible solutions compared to the loiter-based approach, which is less sensitive to obstacle avoidance and dynamic constraints.

Furthermore, in the simulation using the 0-1 costmap with Gaussian distributions, the first path was generated in 75.24 seconds, while the second path was found in 41.38 seconds. These results indicate the algorithm's capability to adapt to varying obstacle densities and generate paths efficiently within the given computational constraints.

Overall, the tests validate the algorithm's robustness in both simulated and real environments, highlighting its potential for online path planning in obstacle rich scenarios. Despite some degradation in tracking accuracy due to real-world challenges, the kinodynamic RRT* algorithm proved to be a reliable tool for generating optimal, collision-free paths.

Finally, Figure 35 illustrates the real fixed-wing aircraft in flight, following

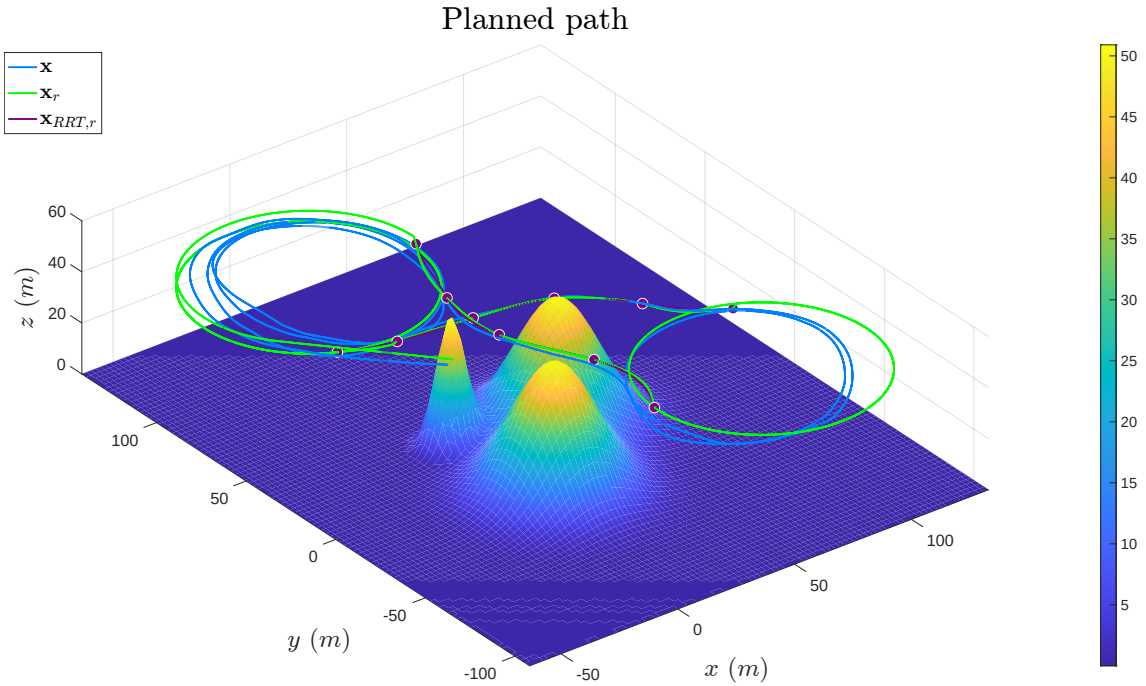

Figure 34: 3D view of the real test with three gaussians. The fixed-wing position is shown in blue, the trajectory in green, and the planned path in purple.

the kinodynamic path generated by the algorithm presented in this work, over Calvert Vaux Park, NYC.

Figure 35: Real fixed-wing aircraft in flight over Calvert Vaux Park, NYC.

Conclusions

This report presents a comprehensive study on online path planning for fixed-wing UAVs, addressing both theoretical and practical challenges. A nonlinear aircraft model has been developed, incorporating feedback linearization techniques to establish a strong foundation for trajectory generation. The application of Kinodynamic RRT* with optimal control strategies ensures that the computed paths are not only kinodynamically feasible but also efficient in terms of arrival time and trajectory smoothness. The system was specifically designed to handle the kinematic and dynamic constraints of fixed-wing UAVs, ensuring that the resulting trajectories are stable and trackable.

The results from MATLAB simulations and simulation along with real-world experiments based on ARPL architecture confirm the viability of the proposed approach, demonstrating its potential for real-time deployment in UAV navigation scenarios. The methodology is particularly well-suited for generating non-aggressive paths in environments with moderate obstacles, ensuring smooth and stable flight. However, it is important to note that achieving a real-time replan within a few seconds remains challenging due to the computational complexity. Nevertheless, it is feasible to compute the next trajectory while executing the previous one, allowing for continuous navigation even in dynamic environments.

In terms of future work, there are several exciting possibilities to enhance the path planning approach further. One potential avenue is exploring multi-agent path planning for collaborative UAVs. In such a system, multiple UAVs could share their path information, dynamically adjusting their trajectories to avoid collisions and optimize mission objectives. Another area of interest is integrating advanced sensor fusion techniques, which could allow the UAVs to better perceive and adapt to environmental changes, such as unpredictable obstacles or shifting wind conditions.

Additionally, further development of adaptive planning strategies, leveraging machine learning or reinforcement learning techniques, could enable the UAVs to optimize their behavior autonomously based on the environment and mission parameters. The use of such adaptive methods could enhance performance in highly dynamic and uncertain environments.

Overall, this study contributes to the advancement of UAV autonomy by providing a robust framework for path planning in constrained and complex operational environments, with an emphasis on generating smooth trajectories that satisfy kinodynamic constraints and enable online computation.

Bibliography

[1] Ren Tianzhu et al. "Three-dimensional path planning of UAV based on an improved A* algorithm*". In: 2016. DOI: 10.1109/CGNCC.2016.7828772.

[2] Dave Ferguson and Anthony Stentz. "Using interpolation to improve path planning: The Field D* algorithm". In: (2006). DOI: 10.1002/rob.20109.

[3] Yong-bo Chen et al. "UAV path planning using artificial potential field method updated by optimal control theory". In: (2016). DOI: 10.1080/00207721.2014.929191.

[4] Fei Yan, Yi-Sha Liu, and Ji-Zhong Xiao. "Path Planning in Complex 3D Environments Using a Probabilistic Roadmap Method". In: (2013). DOI: 10.1007/s11633-013-0750-9.

[5] Steven M. LaValle. *Rapidly-Exploring Random Trees: A New Tool for Path Planning*. Tech. rep. 1998. URL: http://msl.cs.uiuc.edu/~lavalle/papers/Lav98c.pdf.

[6] Sertac Karaman and Emilio Frazzoli. "Sampling-based algorithms for optimal motion planning". In: (2011). DOI: 10.1177/0278364911406761.

[7] Salvatore Rosario Bassolillo et al. "Path Planning for Fixed-Wing Unmanned Aerial Vehicles: An Integrated Approach with Theta* and Clothoids". In: (2024). DOI: 10.3390/drones8020062.

[8] John Valasek. "Small Unmanned Aircraft: Theory and Practice". In: (2013). DOI: 10.2514/1.61067.

[9] James Keller et al. "Obstacle avoidance and path intersection validation for UAS: A B-spline approach". In: 2016. DOI: 10.1109/ICUAS.2016.7502631.

[10] Dustin J. Webb and Jur van den Berg. "Kinodynamic RRT*: Asymptotically optimal motion planning for robots with linear dynamics". In: 2013. DOI: 10.1109/ICRA.2013.6631299.

[11] Jiahao Ge et al. "Trajectory Planning of Fixed-wing UAV Using Kinodynamic RRT Algorithm". In: 2020. DOI: 10.1109/ICIST49303.2020.9202213.

[12] Kiril Solovey et al. "Revisiting the Asymptotic Optimality of RRT*". In: 2020. DOI: 10.1109/ICRA40945.2020.9196553.

[13] Frank L. Lewis, Draguna Vrabie, and Vassilis L. Syrmos. *Optimal Control*. 2012. ISBN: 978-0-470-63349-6.

[14] Thomas H. Cormen et al. *Introduction to Algorithms*. 2009. ISBN: 978-0-262-03384-8.

[15] Jacky Liu. *Comparison of Breadth-First Search and Depth-First Search Methods and Their Uses*. 2024. URL: https://www.linkedin.com/pulse/comparison-breadth-first-search-depth-first-methods-uses-liu-ud4ce.

[16] Luca Morando et al. "Trajectory Planning and Control for Differentially Flat Fixed-Wing Aerial Systems". In: 2025. URL: https://arxiv.org/abs/2502.00581.

[17] J. Hauser and R. Hindman. "Aggressive flight maneuvers". In: 1997. DOI: 10.1109/CDC.1997.649490.